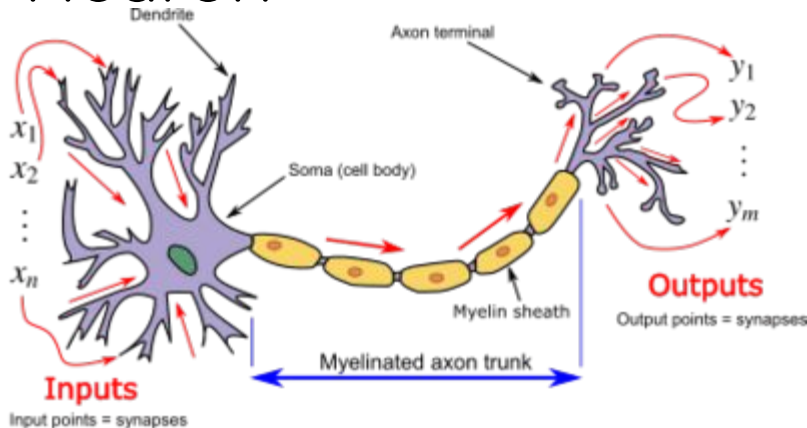


Computer vision

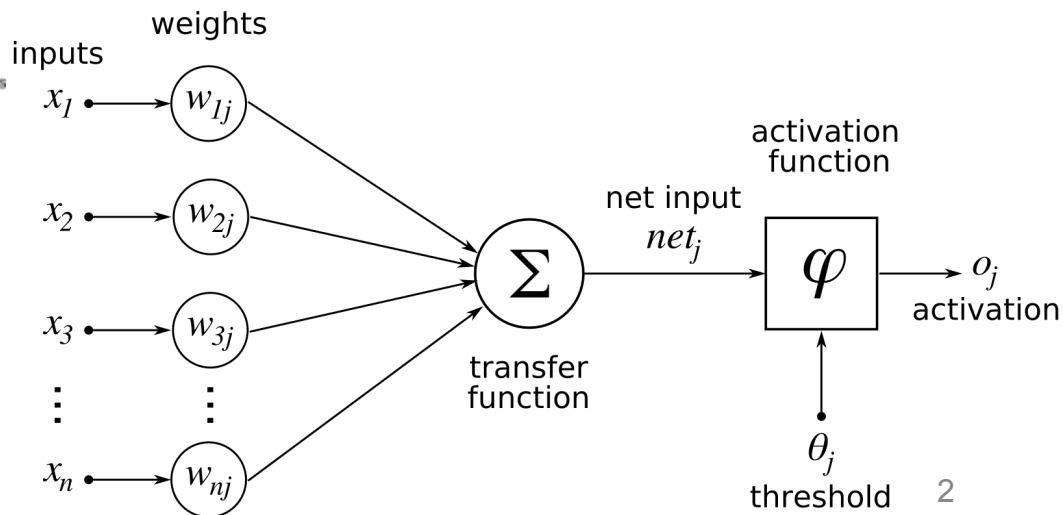
using Deep Neural Networks (DNN)

Model of an artificial neuron

Neuron



Artificial neuron

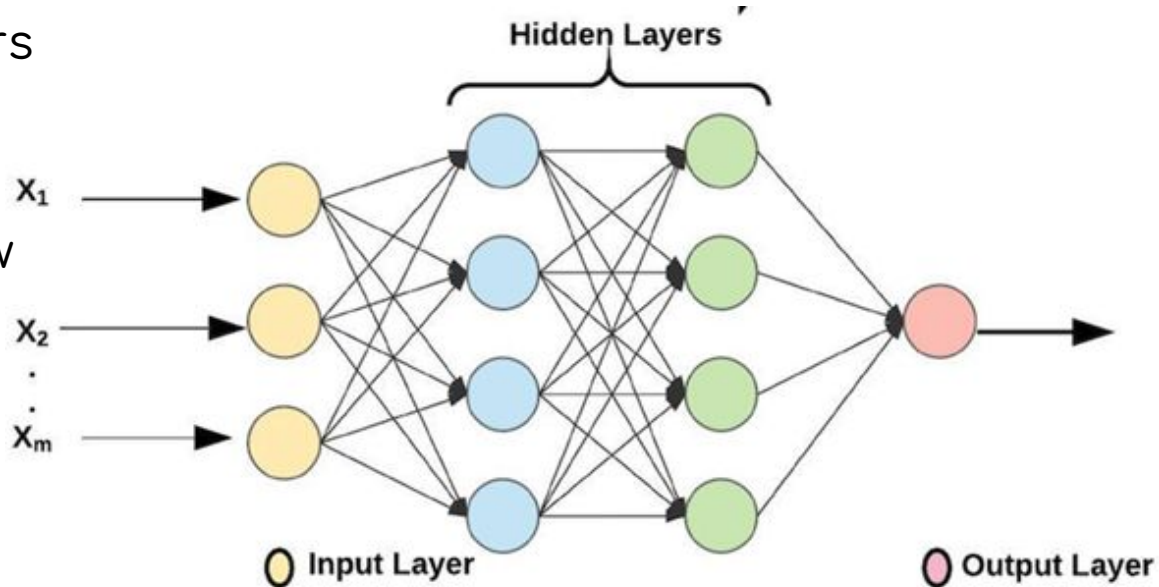


Simple Neural Network (NN)

MLP Multilayer Perceptron

Multi-Layer Perceptron (MLP)
diagram with 2 hidden layers

Neural networks viewed as
directed graph - Signal flow
graph

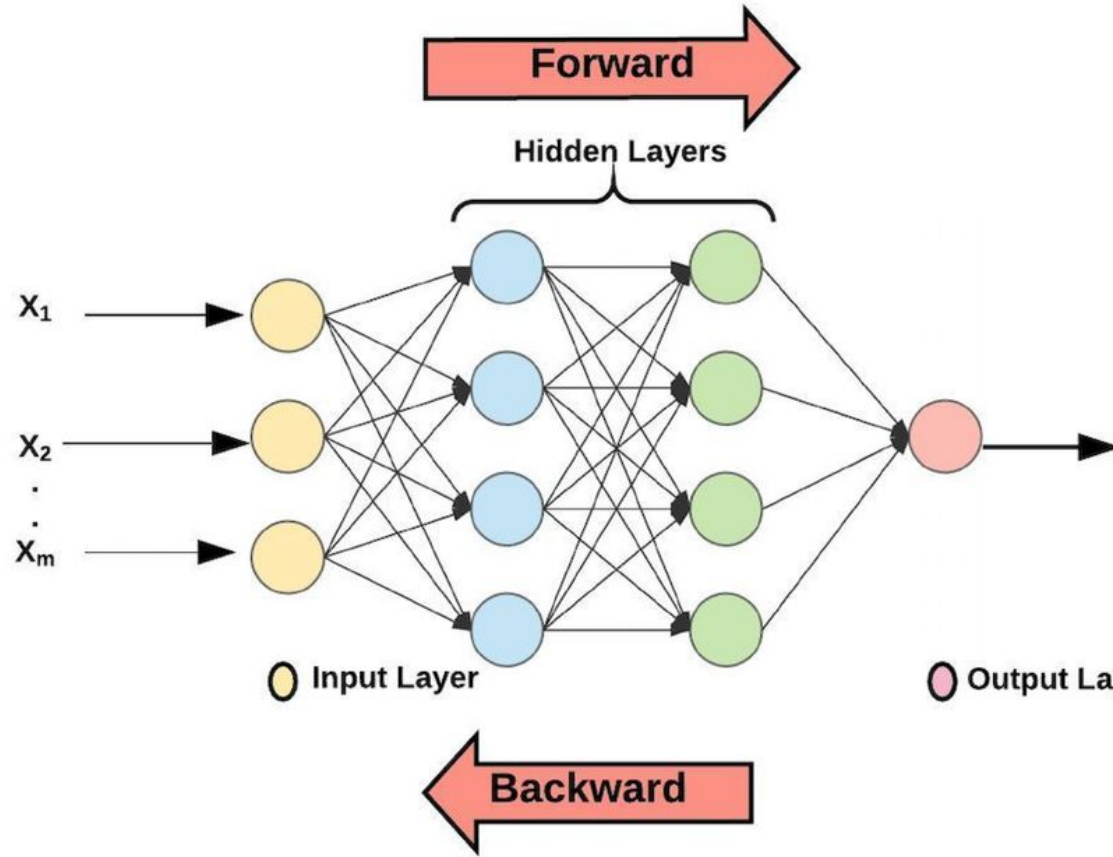


Forward & Backward propagation in NN

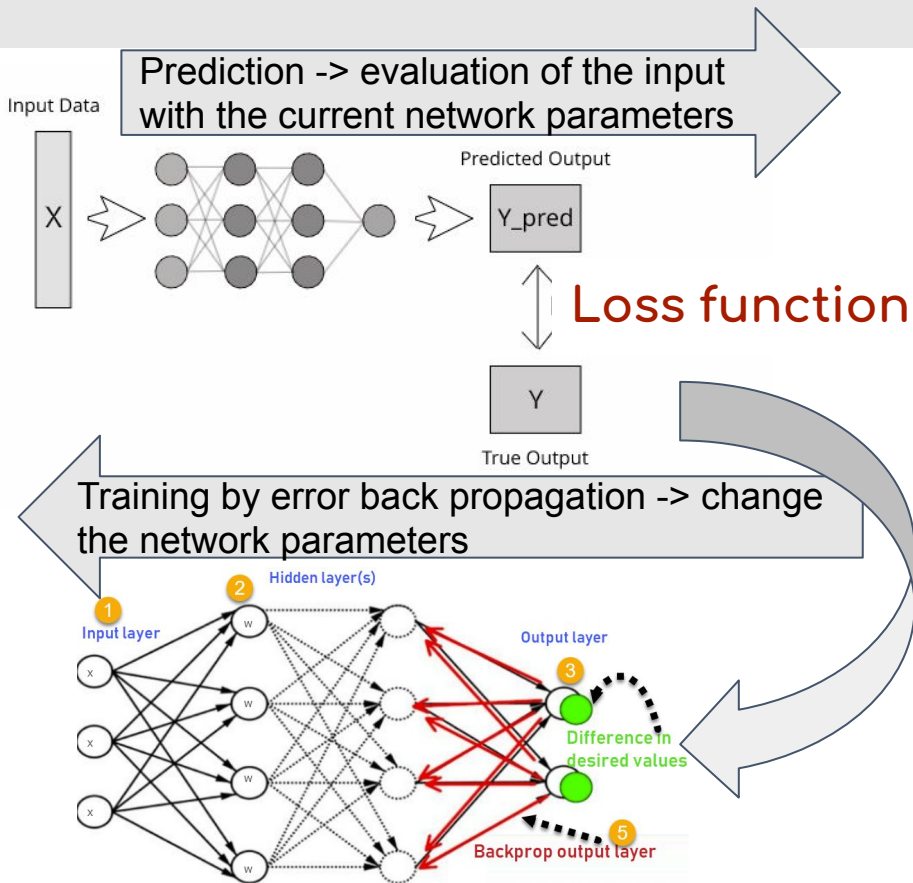
Forward propagation

fundamental process in NN where input data passes through multiple layers to generate an output.

Error Backward Propagation or Backpropagation of Errors, is a key algorithm used to train NN by minimizing the difference between predicted and actual outputs.



Supervised learning - training of NN



- The network learns by comparing the input data and the corresponding expected output data.
- **Loss function** is used for the **evaluation of the prediction error**.
- Successful learning gradually-iteratively reduces the prediction error by using the error back propagation algorithm

Deep Learning

Deep Neural Network DNN

Deep Learning

Deep NN vs. Shallow NN

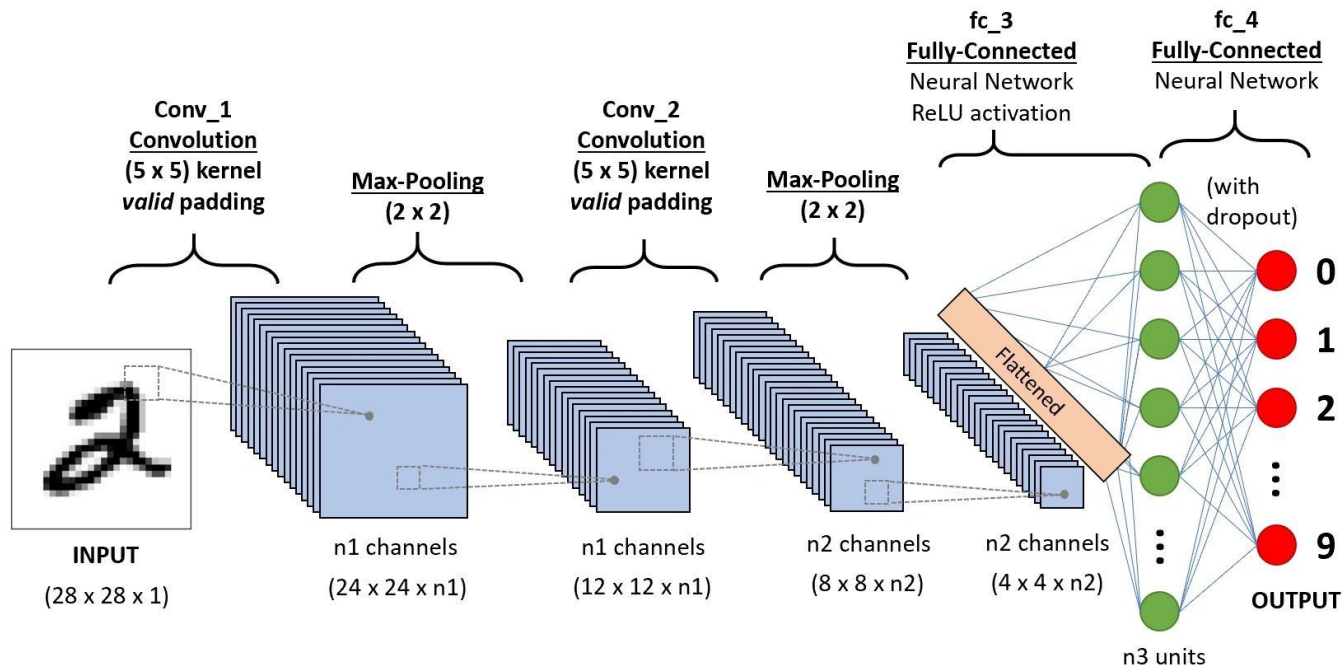
The word "deep" in Deep Learning refers to the number of hidden layers i.e. depth of the neural network.

Every neural network with more than three layers:
one Input Layer, one Hidden layer and one Output Layer can be considered as a Deep Learning Model.

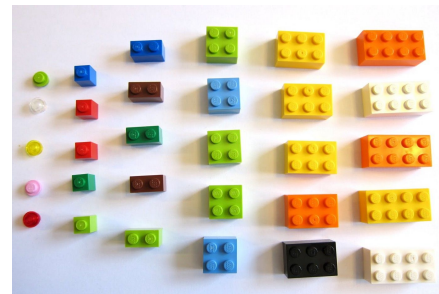
Rem:

This is a simple, but not strict definition; the distinction between deep neural networks and shallow neural networks may vary across different publications.

CNN for MNIST dataset digit recognition



Demo



Convolutional Neural Network CNN

CNN story: 2012

Imagenet database: 14 M labeled images, 20K categories

Dog, domestic dog, Canis familiaris

A member of the genus Canis (probably descended from the common wolf) that has been domesticated by man since prehistoric times; occurs in many breeds; "the dog barked all night!"

1603 pictures 88.15% Popularity Percentile WordNet IDs

Numbers in brackets: (the number of synsets in the subtree)

ImageNet 2011 Fall Release (21641)

- animal, animate being, beast, brute, creature, fauna
- chordate (2953)
 - tunicate, urochordate, urochord
 - cephalochordate (1)
 - vertebrate, craniate (2943)
 - mammal, mammalian (11)
 - metatherian (36)
 - fleshy-mammal (3)
 - placental, placental mammal (10)
 - livestock, stock, far
 - hyrax, coney, cony, Unguicula (0)
 - bat, chiropteran (38)
 - pachyderm (8)
 - pangolin, scaly ante
 - digitigrade mamma
 - carnivore (362)
 - bear (11)
 - musteline mammal (8)
 - procyonid (8)
 - viverrine, viverrid
 - canine, canid (2)
 - wild dog (5)
 - hyena, hyae
 - bitch (1)
 - jackal, Canis
 - fox (11)
 - wolf (5)

Treemap Visualization **Images of the Synset** **Downloads**

ImageNet 2011 Fall Release > Canine, canid > Dog, domestic dog, Canis familiaris

Hunting **Working**

Toy **Great** **Puppy** **Griffon**

Dalmatian **Basenji** **Corgi**

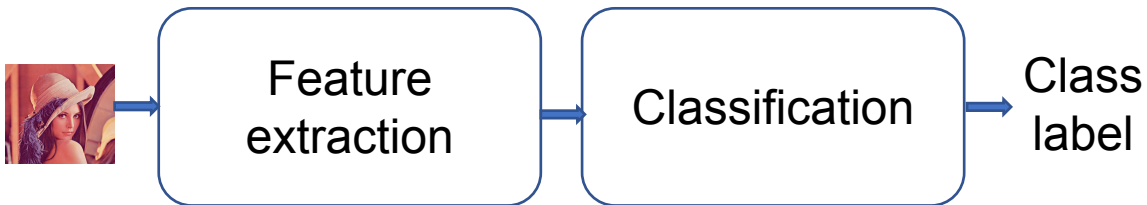
Lapdog **Poodle** **Mexican**

Pooch **Leonberg** **Newfou**

Cur **Spitz**

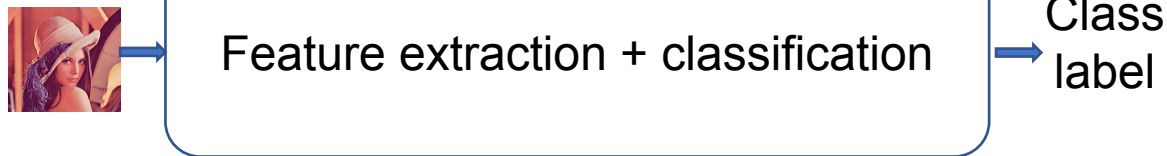
ImageNet Large Scale Visual Recognition Challenge (ILSVRC)

Classification task and Deep Learning



- Traditional approach

Suboptimal, requires expert knowledge, hand crafted features

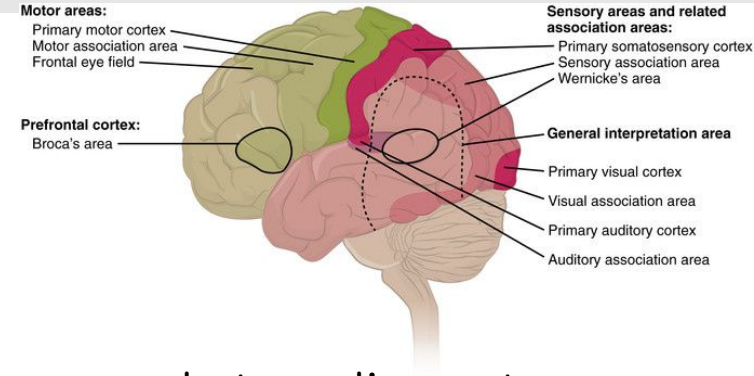


- CNN - Deep learning

both the classifiers and the features are learned automatically

Convolutional Neural Network Inspired by biology

Visual cortex - topographical representation
of the field of vision



- Reception field of neighbouring cells corresponds to adjacent portions of the field of vision
- Sequence of cells can perceive complex shapes
- Uses convolution as building blocks

2D cross-correlation & convolution-recap.

Correlation

$$\text{Corr}(f(x, y) \ g(x, y)) = \sum_{m=0}^{M-1} \sum_{n=0}^{N-1} f(m, n) g(x + m, y + n)$$

Convolution

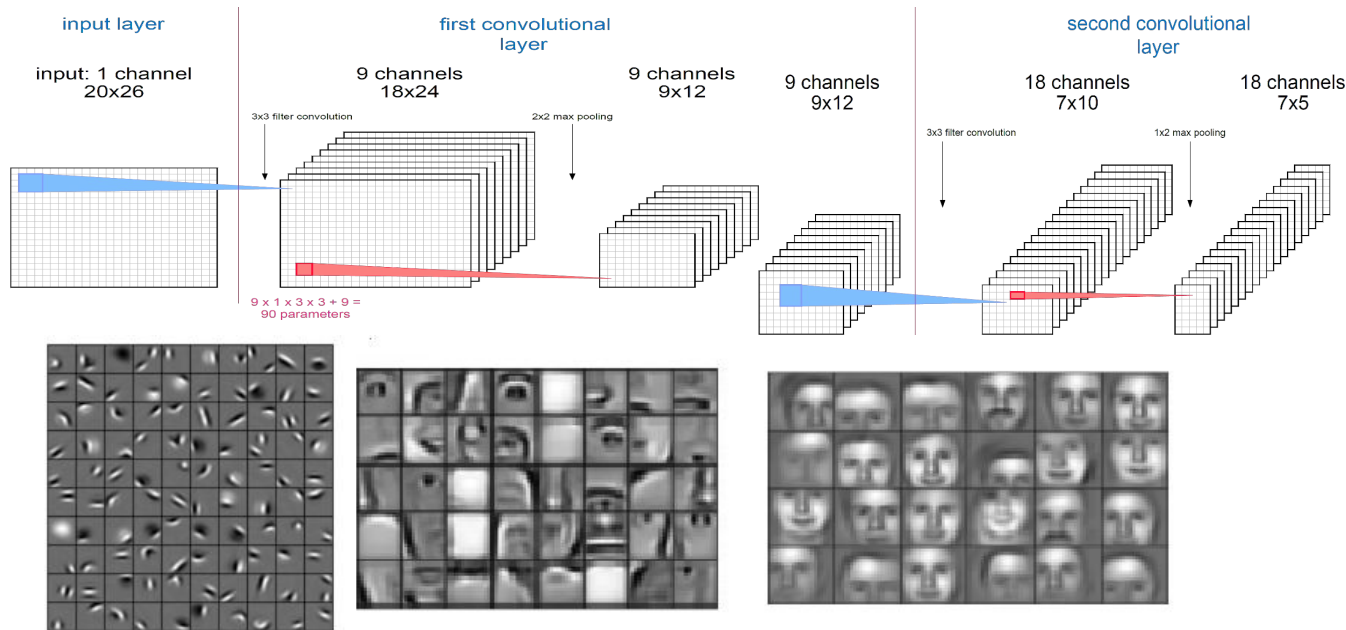
$$f(x, y) * g(x, y) = \sum_{m=0}^{M-1} \sum_{n=0}^{N-1} f(m, n) g(x - m, y - n)$$

The kernel is learned -> we do not flip the kernel ->

Cross-correlation is easier to implement for learned kernels.

We call it “Convolution” 😊

Convolutional layers in CNN - example



Learned hierarchical features from a deep learning algorithm.

Hyperparameters vs parameters of NN

- A model **hyperparameter** is the parameter whose value **is set before** the model start training.
 - hyperparameters cannot be learned
 - hyperparameters are set manually
- A model **parameter** is a variable of the selected model which will be set by learned from data.
 - They are not set manually
 - The final parameters found after training will decide how the model will perform on unseen data

Architecture of CNN

Layers in CNN

Layers in CNN

Input layer

Convolution layer (Convol.)

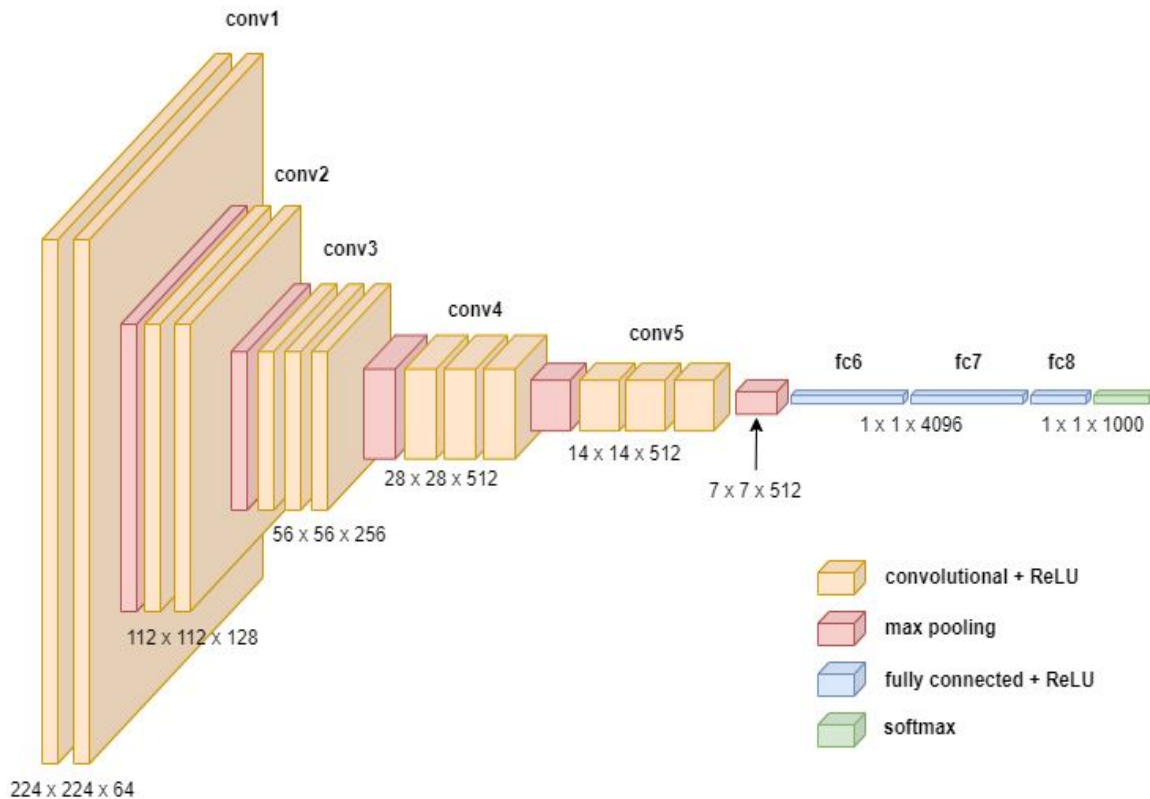
Activation functions (ReLU)

Pooling layer (Max pooling)

Fully Connected (Fully con)

Softmax

Output layer



Input layer: vector or matrix

1D CNN:

Input vector of data

2D CNN for image processing:

Input matrix of data: Raw image data or preprocessed image data

Input image could be a grayscale image - 1 channel
or multichannel image -> color image represented by 3 channels (R, G, B).

Example: An RGB image of size 128×128 pixels has an input **tensor** shape of (128, 128, 3).

Convolution Layer

- core building block of CNNs.

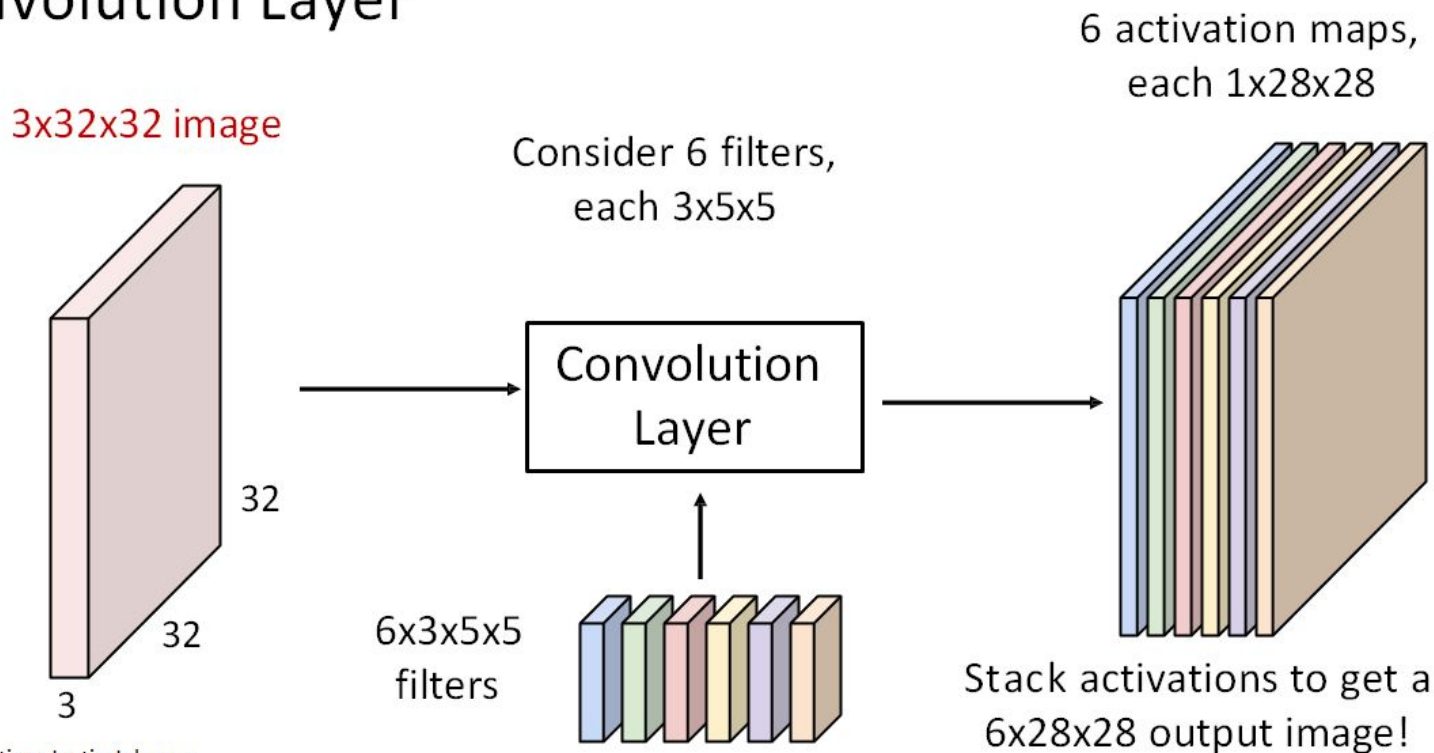
Extracts local spatial features such as edges, textures, corners, and patterns using learnable filters (kernels)

Each filter is matrix/tensor of trainable weights

- Each filter (a small matrix, e.g., 3×3 or 5×5) slides across the image and performs a convolution operation, computing the dot product between the filter and the image region.
- each convolution layer is composed by a configurable number of filters
- convolution operation is performed between the image and each filter, producing as output a new tensor

Convolutional Layer -example

Convolution Layer



Padding in Convolutional Layer

0	0	0	0	0	0	0	0
0	3	3	4	4	7	0	0
0	9	7	6	5	8	2	0
0	6	5	5	6	9	2	0
0	7	1	3	2	7	8	0
0	0	3	7	1	8	3	0
0	4	0	4	3	2	2	0
0	0	0	0	0	0	0	0

$6 \times 6 \rightarrow 8 \times 8$

*

1	0	-1
1	0	-1
1	0	-1

3×3

=

-10	-13	1			
-9	3	0			

6×6

Hyperparameter

Example of zero padding $p=1$: applying padding of 1 before convolving with 3×3 filter

Downsampling the Size of Feature Maps in Subsequent Conv. Layers

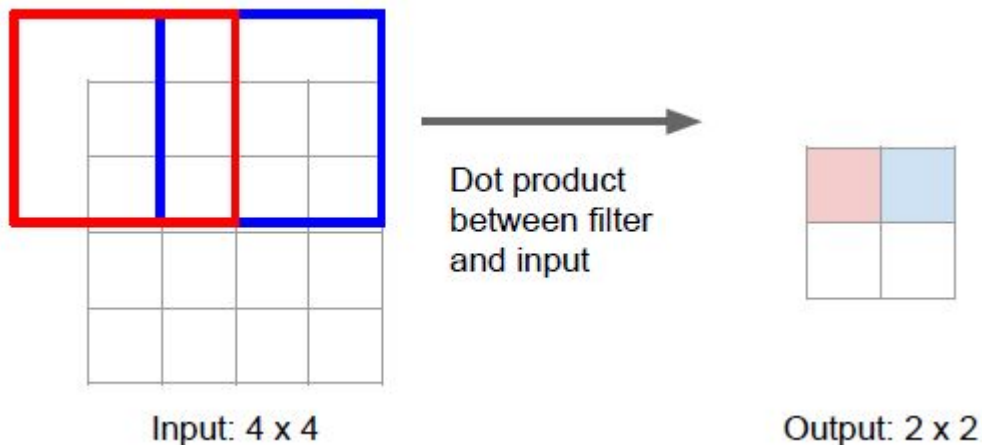
For the purpose of downsampling can serve:

- strided convolution
or
- pooling layer

It depends on the model architecture and the dataset used and the task performed.

Strided convolution

example: 3 x 3 convolution, stride 2 pad 1



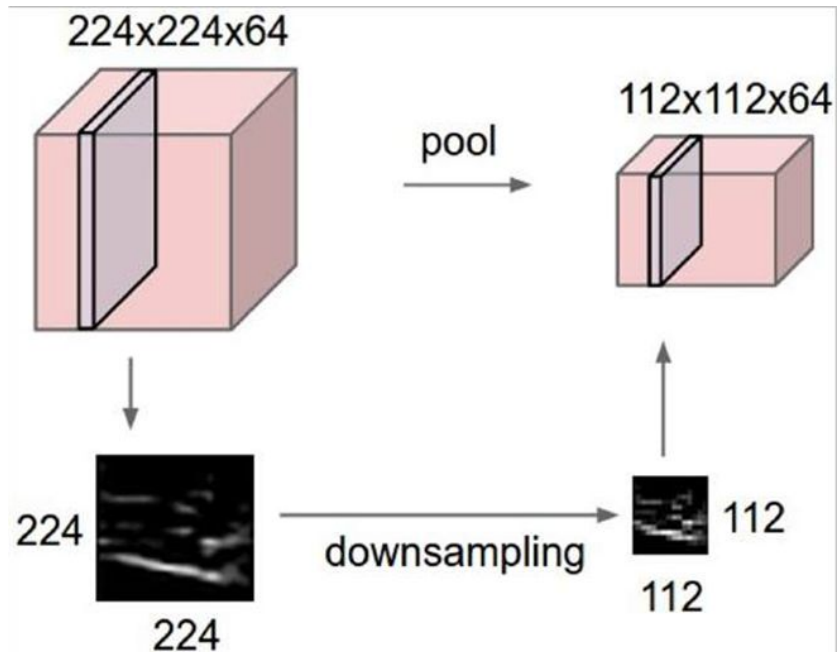
Filter moves 2 pixels in the input for every one pixel in the output

Stride gives ratio between movement in input and output

Pooling layer

Pooling layer is used in CNNs to reduce the spatial dimensions (width and height) of the input feature maps while retaining the most important information.

- **Decreases the number of parameters:** Pooling layers reduce the spatial size of the feature maps, which decreases the number of parameters and computations in the network.
- **Build a hierarchical representation of features,** where lower layers capture fine details and higher layers capture more abstract and global features.



Basic types of pooling layers

Max pooling

2	2	7	3
9	4	6	1
8	5	2	4
3	1	2	6

Max Pool
→
Filter - (2 x 2)
Stride - (2, 2)

9	7
8	6

Average pooling

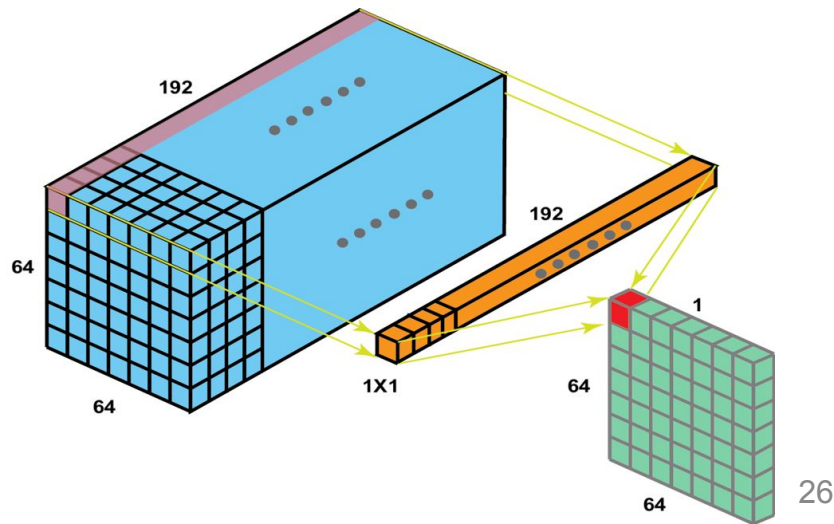
2	2	7	3
9	4	6	1
8	5	2	4
3	1	2	6

Average Pool
→
Filter - (2 x 2)
Stride - (2, 2)

4.25	4.25
4.25	3.5

1x1 convolution

A 1x1 convolution is a convolutional neural network operation that applies a 1x 1 kernel across the input, essentially acting as a mini-neural network at each pixel's location **to combine features across channels**.



Activation Layer : Non-Linearity Layer

Every value in the tensor is fed through an activation function to introduce a nonlinearity.

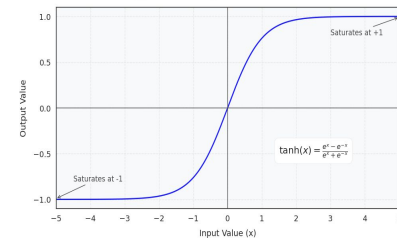
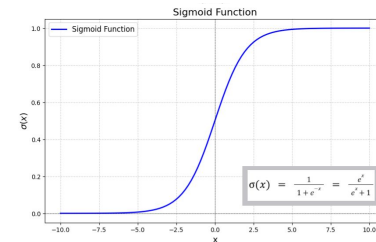
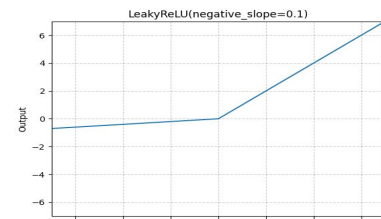
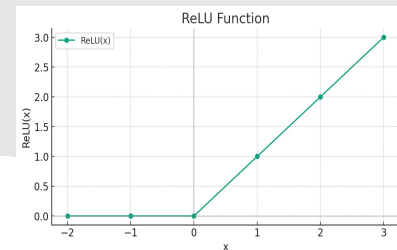
If you remove the activation functions, the entire network becomes a stack of linear operations.

Real-world image data contain non-linear relationships and linear operations alone cannot model these complexities.

Activation functions allow CNNs to build hierarchical, non-linear representations.

Common type of activation functions

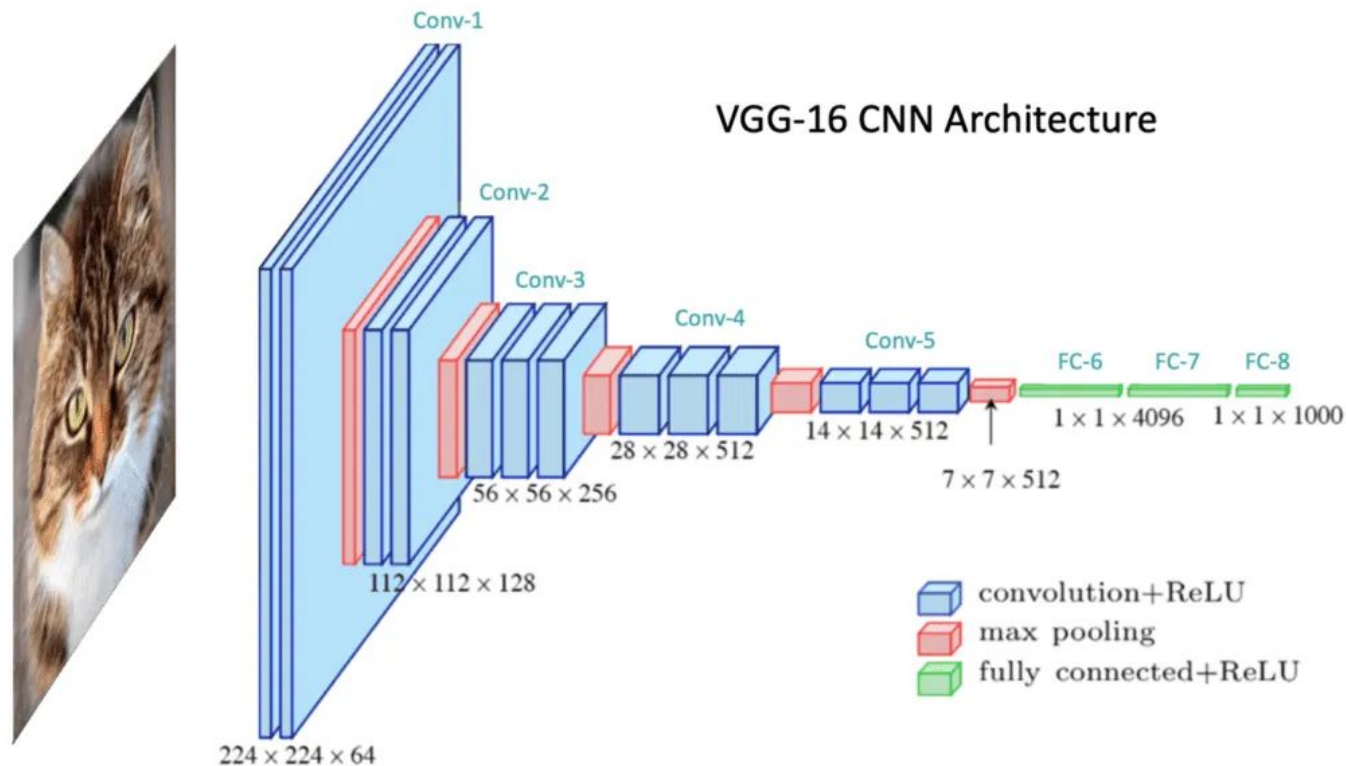
Function	Formula	Key Property
ReLU (Rectified Linear Unit)	$f(x) = \max(0, x)$	Fast, prevents vanishing gradient
Leaky ReLU	$f(x) = x$ if $x > 0$, else $0.01x$	Avoids dead neurons
Sigmoid	$f(x) = \frac{1}{1 + e^{-x}}$	Maps to $[0, 1]$, used in binary output
Tanh	$f(x) = \tanh(x)$	Zero-centered output



Hyperparameters of Convolution layer

- **Number of filters:** The number of convolutional filters in the layer
- **Width of filter:** The width of the filter matrix of a single filter.
- **Height of filter:** The width of the weight matrix of a single filter.
- **Horizontal stride:** The number of pixels to move while performing the convolution along the horizontal axis.
- **Vertical stride:** The number of pixels to move while performing the convolution along the vertical axis.
- **Padding:** Same results in padding the input such that the output has the same length as the original input.
- **Activation function:** The function that will applied to each element of the output. (ReLU, sigmoidal...)

Example of DNN architecture



Fully Connected Layer/Layers

Fully connected layers, also known as dense layers, connect every neuron in a layer to every neuron in previous and subsequent layers

A fully connected layer multiplies the input by a weight matrix and then adds a bias vector.

-> output value is given by activation of the output neuron

Demo

ConvNetJS CIFAR-10 demo

Description

This demo trains a Convolutional Neural Network on the [CIFAR-10 dataset](#) in your browser, with nothing but Javascript. The state of the art on this dataset is about 90% accuracy and human performance is at about 94% (not perfect as the dataset can be a bit ambiguous). I used [this python script](#) to parse the [original files](#) (python version) into batches of images that can be easily loaded into page DOM with img tags.

This dataset is more difficult and it takes longer to train a network. Data augmentation includes random flipping and random image shifts by up to 2px horizontally and vertically.

By default, in this demo we're using Adadelta which is one of per-parameter adaptive step size methods, so we don't have to worry about changing learning rates or momentum over time. However, I still included the text fields for changing these if you'd like to play around with SGD+Momentum trainer.

Report questions/bugs/suggestions to [@karpathy](#).



<http://cs.stanford.edu/people/karpathy/convnetjs/demo/cifar10.html>

Learning of CNN

Supervised learning - two steps:

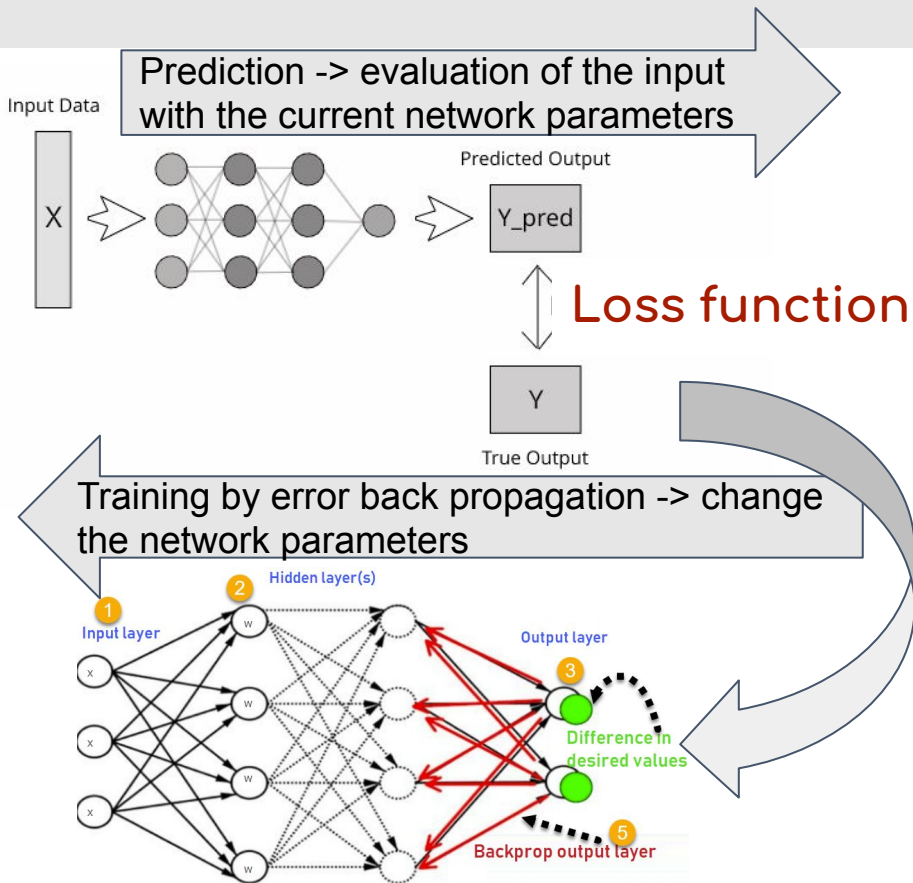
1. Forward Pass Work

The input data are fed into the input of the neural network, and the corresponding output values are computed with the current NN parameters.

2. Backward Pass

In the backward pass the error (the difference between the predicted and actual output) is propagated back through the network to adjust the weights and biases.

Supervised learning - training of NN

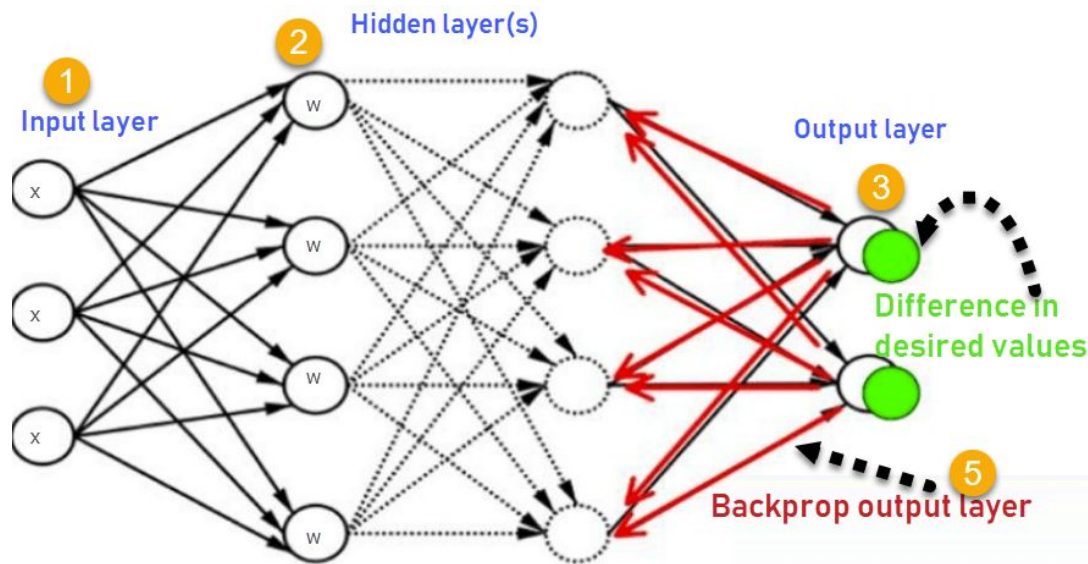


- The network learns by comparing the input data and the corresponding expected output data.
- **Loss function** is used for the **evaluation of the prediction error**.
- Successful learning using error back propagation gradually-iteratively reduces the prediction error

Error Backpropagation

Computing gradients of expressions through recursive application of chain rule

(Řetízkové pravidlo, řetězové pravidlo neboli pravidlo o derivaci složené funkce)



Loss function & Gradient descent

Loss function vs. cost function

The **loss function** is used to evaluate the error between the predicted value and the true value **for a single record**.

The **cost function** measures the **model's error across a group of records** (i.e., the entire dataset)

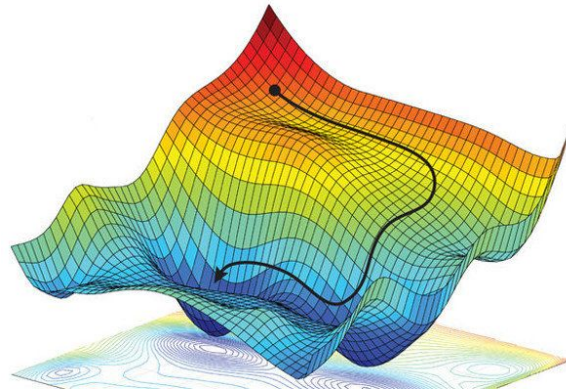
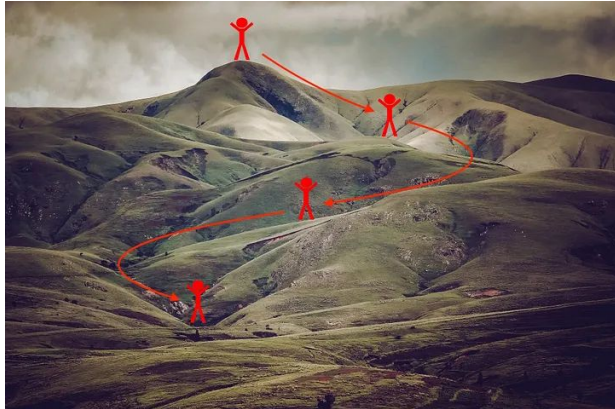
Selected Loss Functions for Classification & Regression

Loss Function	Classification	Regression	
Mean Square Error (MSE) / L2 Loss	✗	✓	$MSE = (1/n) * \sum (y_i - \bar{y})^2$
Mean Absolute Error (MAE) / L1 Loss	✗	✓	$MAE = (1/n) * \sum y_i - \bar{y} $
Huber Loss / Smooth Mean Absolute Error	✗	✓	$L(\delta, y, f(x)) = \begin{cases} (1/2) * (f(x) - y)^2 & \text{if } f(x) - y \leq \delta \\ \delta * f(x) - y - (1/2) * \delta^2 & \text{if } f(x) - y > \delta \end{cases}$
Binary Cross-Entropy Loss / Log Loss	✓	✗	$L(y, f(x)) = -[y * \log(f(x)) + (1 - y) * \log(1 - f(x))]$
Categorical Cross-Entropy Loss	✓	✗	
Hinge Loss	✓	✗	$L(y, f(x)) = \max(0, 1 - y * f(x))$
Log Loss	✓	✗	

Update of weights & Gradient descent

Training: we wish that the loss function decrease ->
update the weight vector in negative gradient direction

Procedure of repeatedly evaluating the gradient and then performing a parameter update is called **Gradient Descent**.

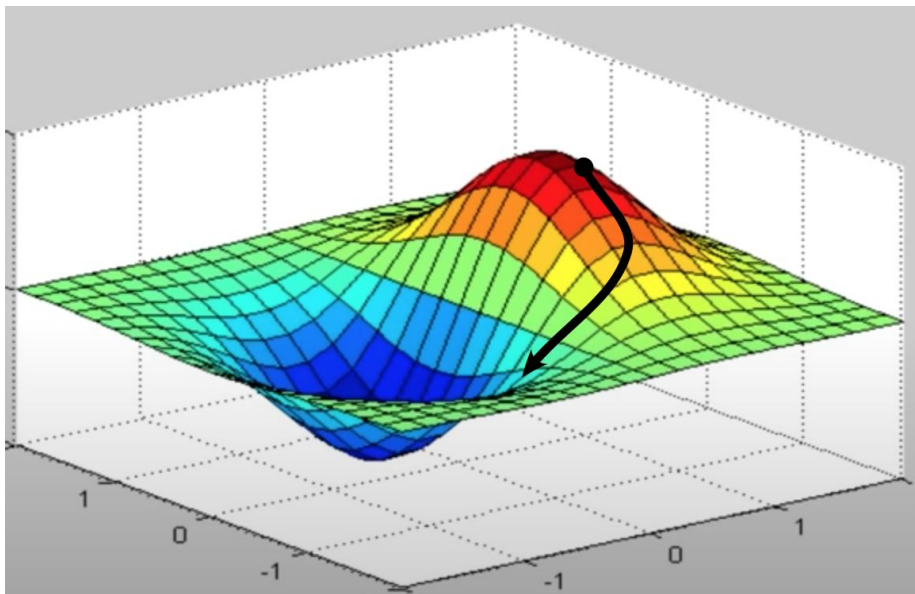


Optimize with Gradient Descent

Gradient Descent is an iterative optimization algorithm used to find the minimum of a differentiable function $f(\theta)$ -typically a loss function in machine learning.

The idea:

Move the parameters θ in the opposite direction of the gradient of the function, because the gradient points toward the direction of maximum increase.



Optimize with Gradient Descent

Let $f(\theta)$ be the loss function we want to minimize.

Then, the update rule is:

$$\theta_{t+1} = \theta_t - \eta \nabla_{\theta} f(\theta_t)$$

where:

θ_t is current parameter vector at iteration t

η is learning rate (a positive scalar step size)

$\nabla_{\theta} f(\theta_t)$ is gradient of f with respect to θ_t

θ_{t+1} is updated parameter vector

Intuitive Meaning: The gradient $\nabla f(\theta_t)$ gives the direction of steepest increase of the function.

Moving in the negative gradient direction reduces the function value - thus, the name descent.

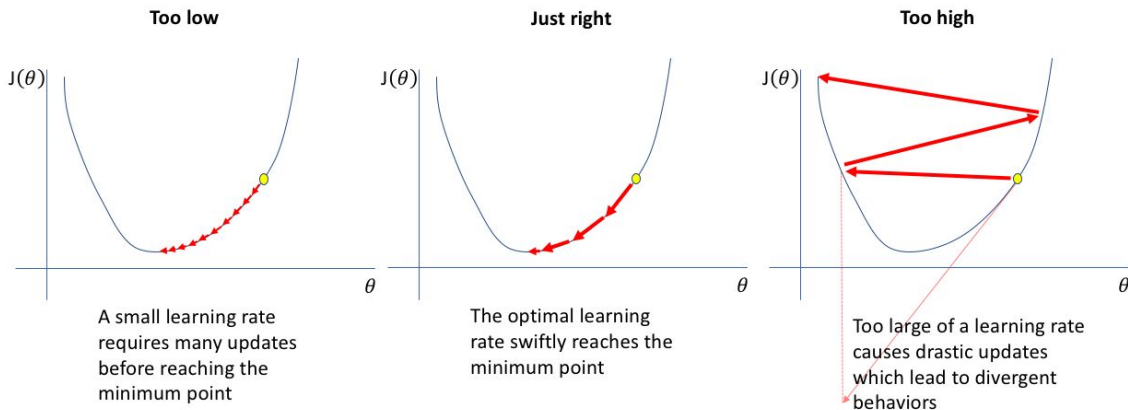
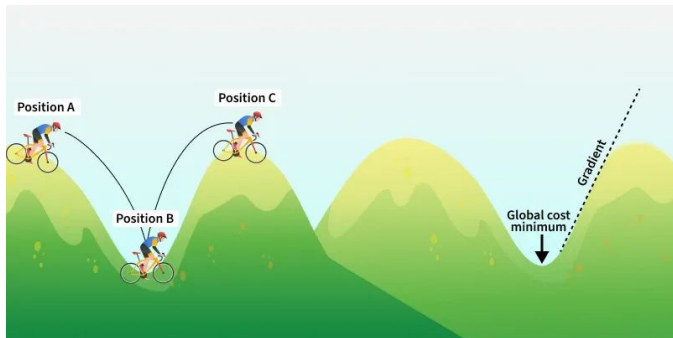
Step size - learning rate η

$$\theta_{t+1} = \theta_t - \eta \nabla_{\theta} f(\theta_t)$$

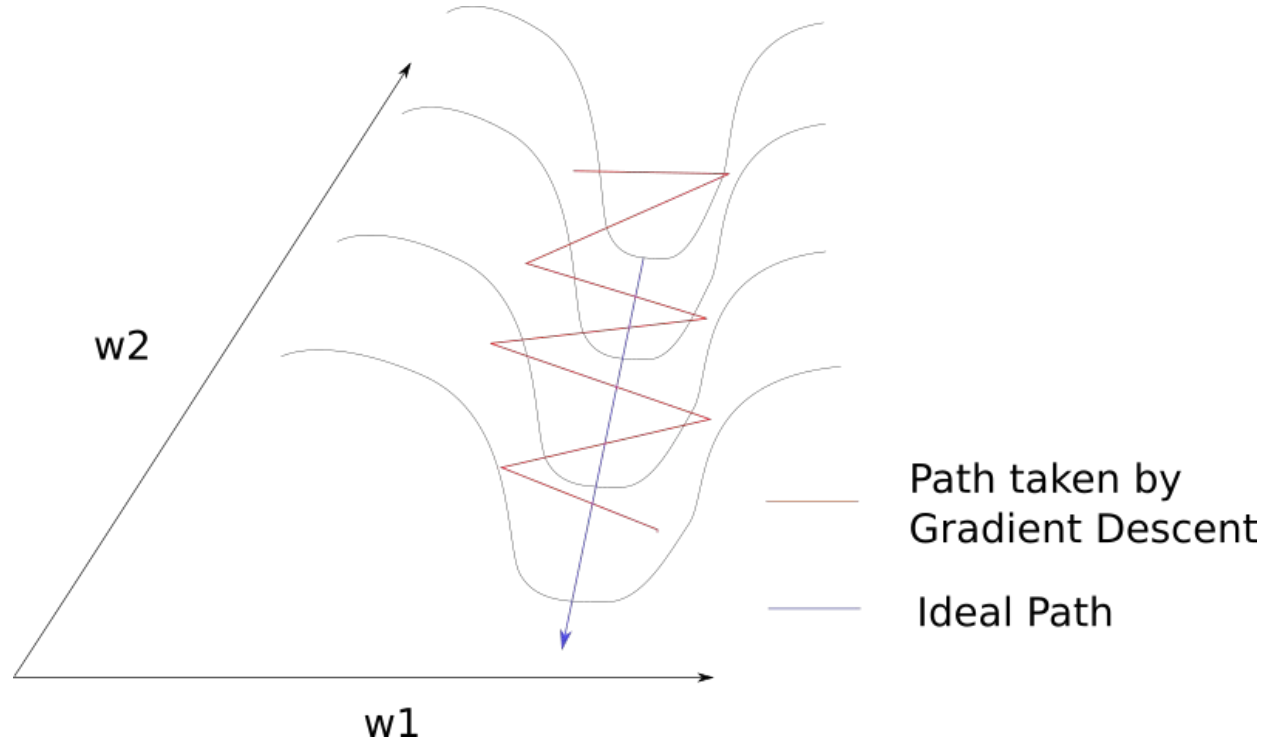
important hyperparameter settings in training a neural network

small step size $\rightarrow$ very small progress

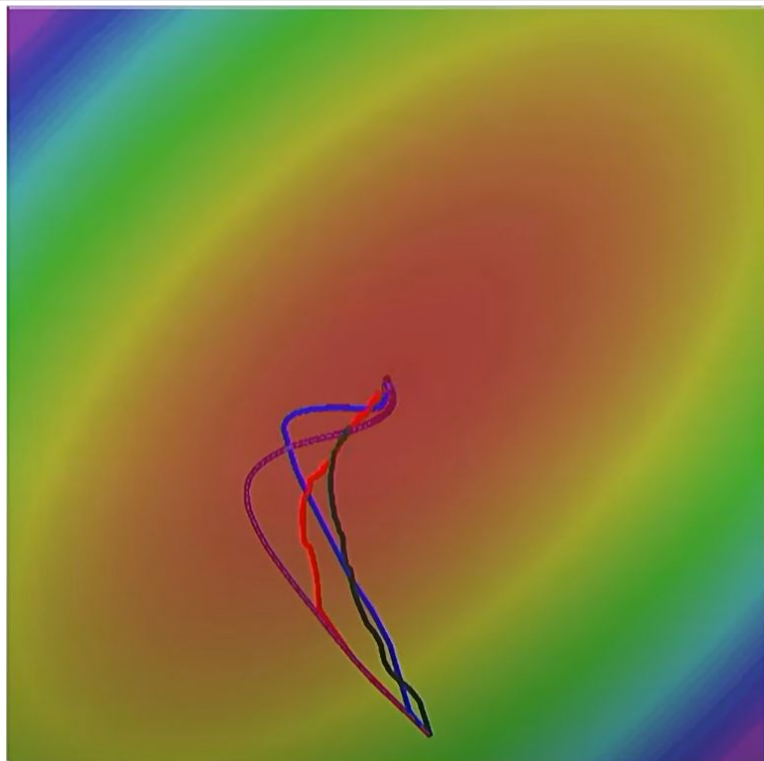
large steps $\rightarrow$ can lead to better progress but are more risky. Note that eventually, for a large step size we will overshoot and make the loss worse.



Problem With Gradient Descent



Examples of common optimisers



— SGD

— SGD+Momentum

— RMSProp

— Adam

Stochastic Gradient
Descent (SGD)

Stochastic Gradient
Descent (SGD) +
Momentum

Root Mean Square
(RMS) Propagation

Adam optimiser

Training data batch

Common approach is to compute the gradient over **batches of the training data**.

Batch size setting - hyperparameter tuning

Why?

- computational efficiency
- improves generalization, preventing overfitting

Batch Size

1. Batch descent
 - batch size = M
2. Mini-batch descent
 - batch size > 1
3. Stochastic descent
 - batch size = 1

Batch Size	
Batch descent	M
Mini-batch descent	BS = 16,32,64,128...
Stochastic descent	1

Samples in mini batch for training must be chosen randomly

Importance of the choice of Loss function

- Guide Model Training:
 - During training, algorithms such as Gradient Descent use the loss function to adjust the model's parameters and try to reduce the error and improve the model's predictions.
- Measure Performance:
 - By finding the difference between predicted and actual values and it can be used for evaluating the model's performance.
- Affect learning behavior:
 - Different loss functions can make the model learn in different ways depending on what kind of mistakes they make.

Epoch

Neural networks are trained in a series of epochs.

Each epoch consists of one forward pass and one backpropagation pass over all of the provided training samples.

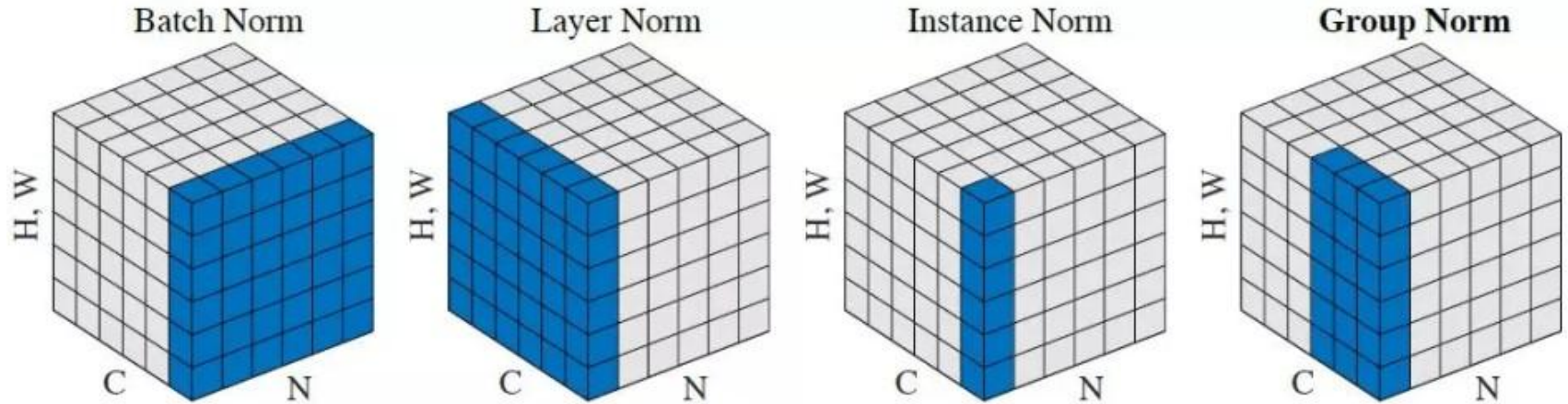
Normalisation

Normalization in a neural network is a technique to scale input data to a standard range, which stabilizes and accelerates the training process.

Common methods include batch normalization, which normalizes activations across a **mini-batch**, and simple **input normalization**, which scales features to a common range like $[0,1]$ or adjusts them to have a mean of 0 and a standard deviation of 1. -> **Z-score normalisation**

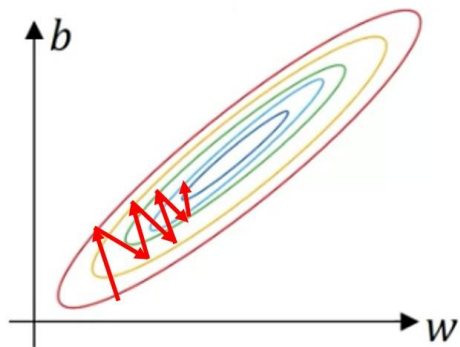
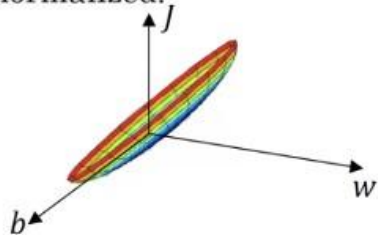
This prevents issues like vanishing or exploding gradients and ensures features with larger scales don't dominate the learning process.

Types of normalization

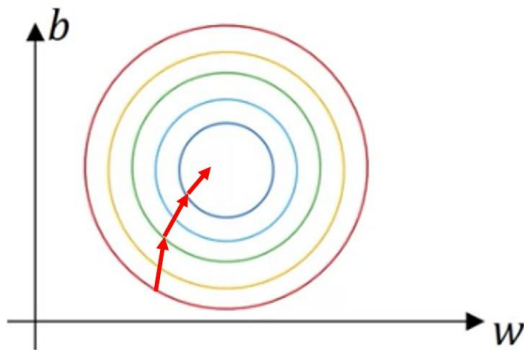
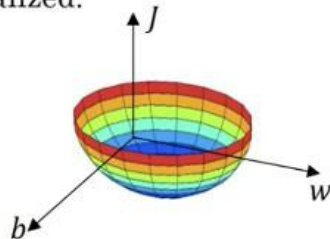


Normalization

Unnormalized:



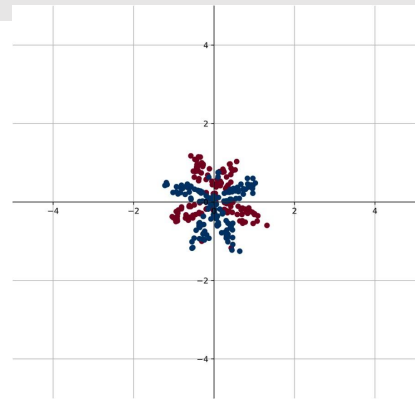
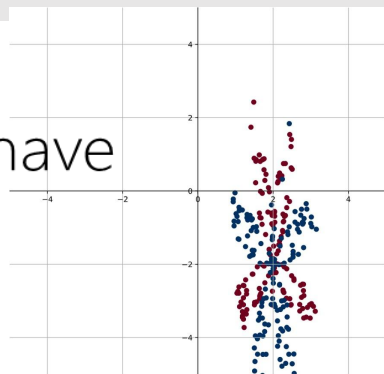
Normalized:



Z-score normalisation

Z-score normalization:

Center and scale a feature to have
mean = 0 and
standard deviation = 1



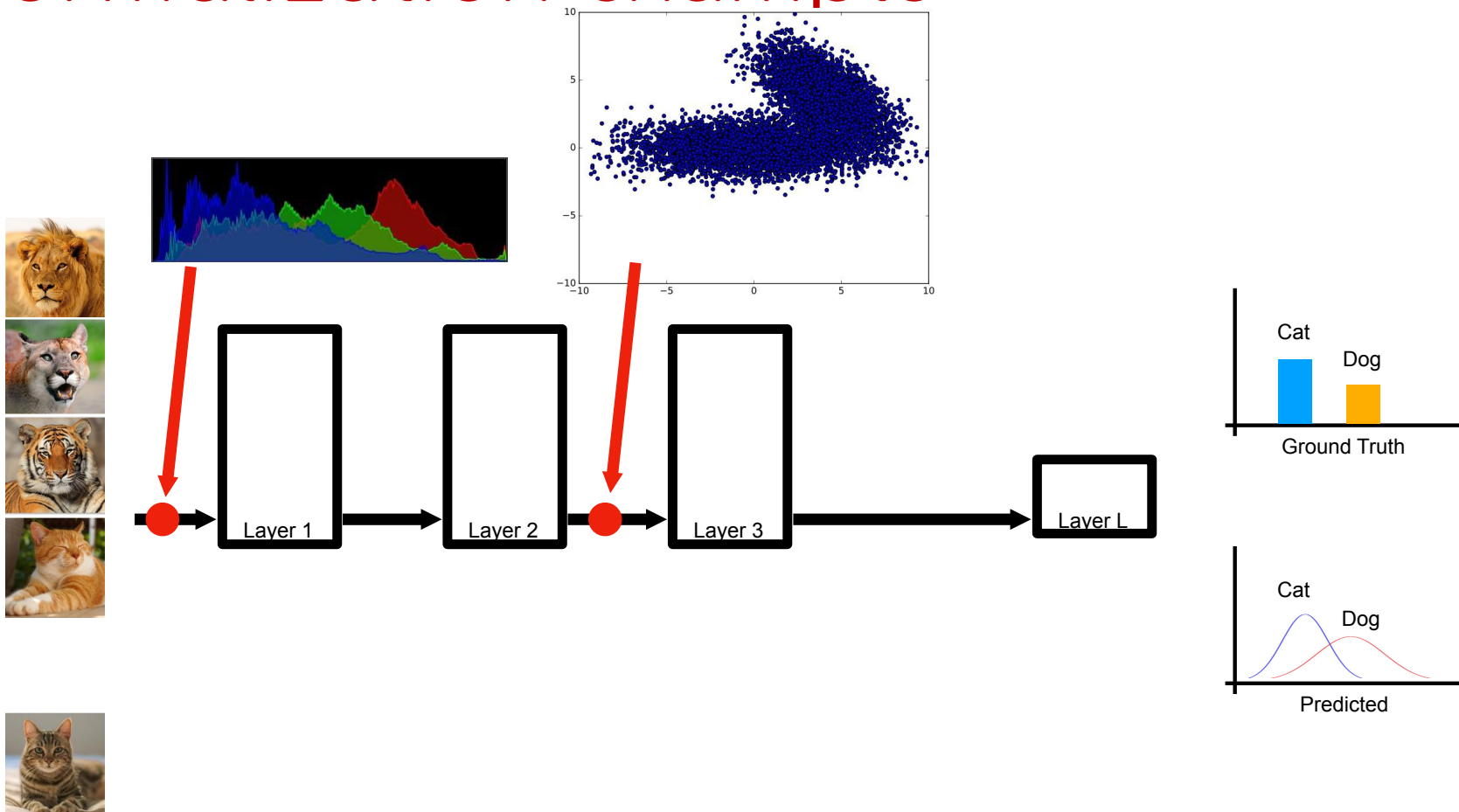
For a random variable X with mean μ and standard deviation σ , the z -score of a value x is :

$$z = \frac{(x - \mu)}{\sigma}$$

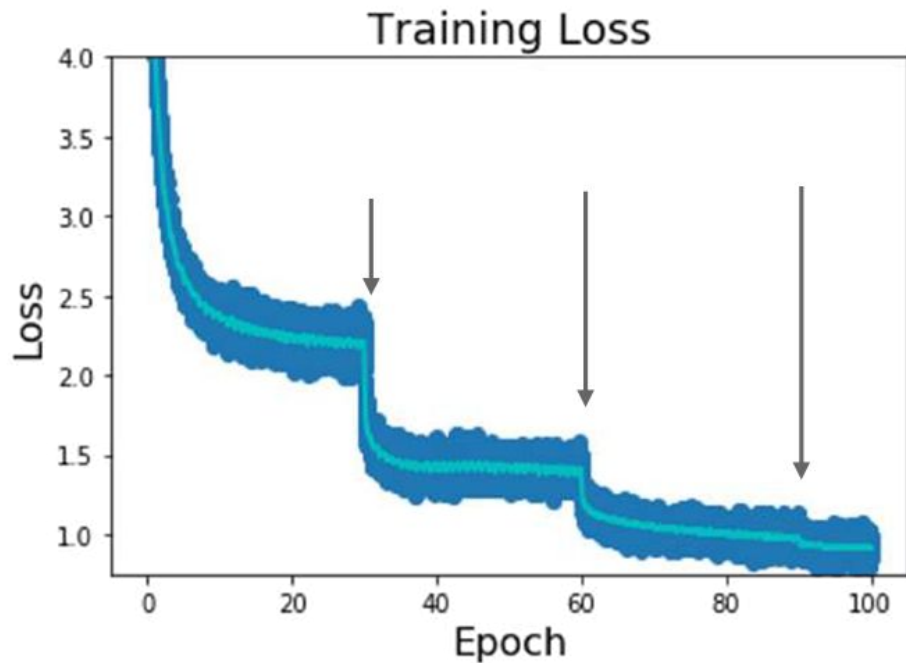
z -scores measure the distance of a data point from the mean in terms of the standard deviation.

The standardized data set has mean 0 and standard deviation 1, and retains the shape properties of the original data set (same skewness and kurtosis).⁵³

Normalization example



Learning rate (LR) scheduling



Step: Reduce learning rate at a few fixed points.

E.g: multiply LR by 0.1 after epochs 30, 60, and 90.

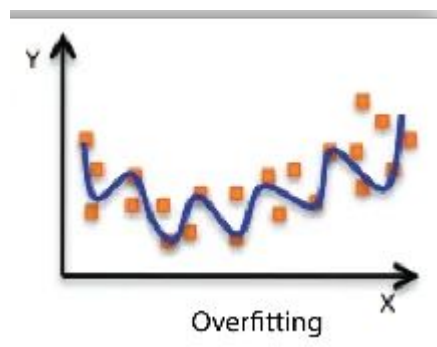
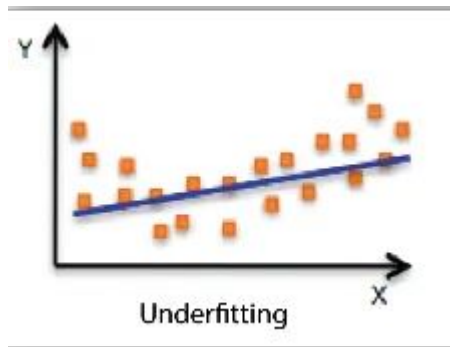
Cosine: $\alpha_t = \frac{1}{2}\alpha_0 (1 + \cos(t\pi/T))$

Linear: $\alpha_t = \alpha_0(1 - t/T)$

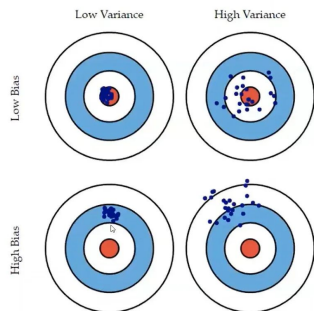
Inverse sqrt: $\alpha_t = \alpha_0/\sqrt{t}$

α_0 : Initial learning rate
 α_t : Learning rate at epoch t
 T : Total number of epochs

Problem of generalization

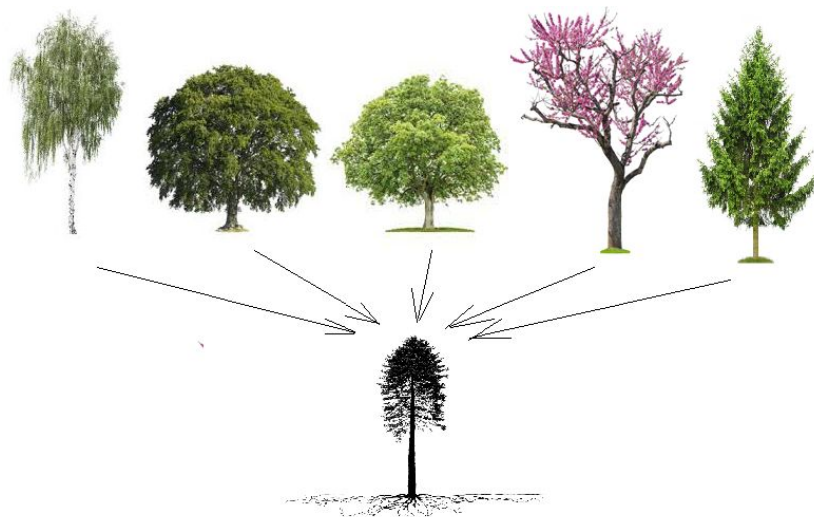
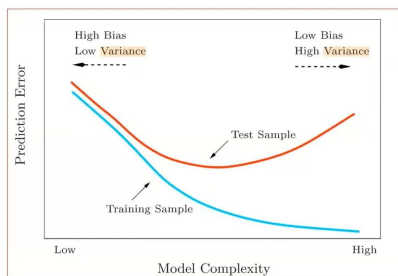


Bias and Variance



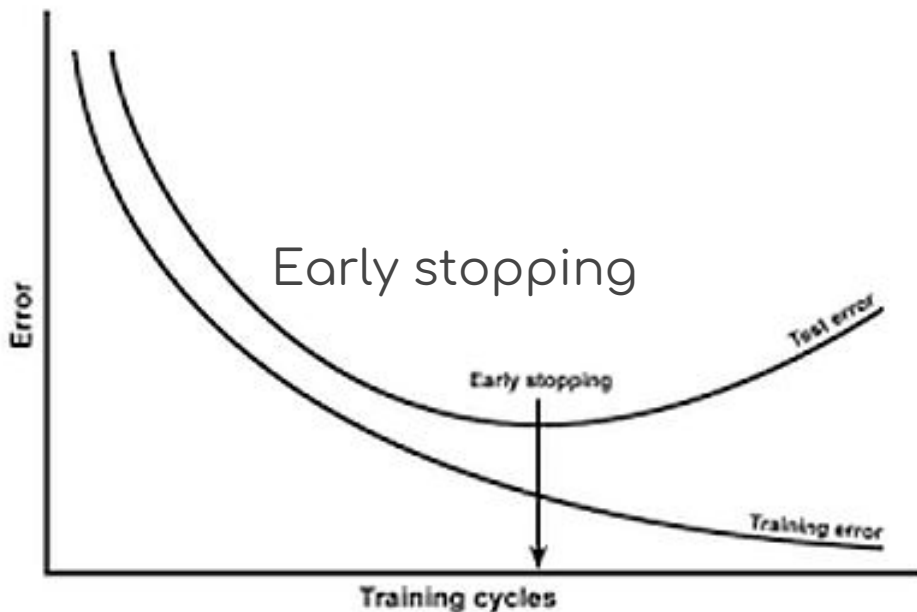
$$\text{Bias} = \mathbb{E}[\hat{f}(x)] - f(x)$$

$$\text{Variance} = \mathbb{E}[(\hat{f}(x) - \mathbb{E}[\hat{f}(x)])^2]$$



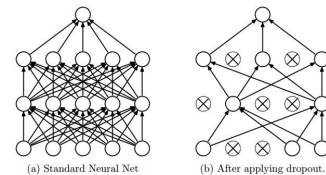
Diagnosing Overfitting

Loss function during the training process



Preventing Overfitting

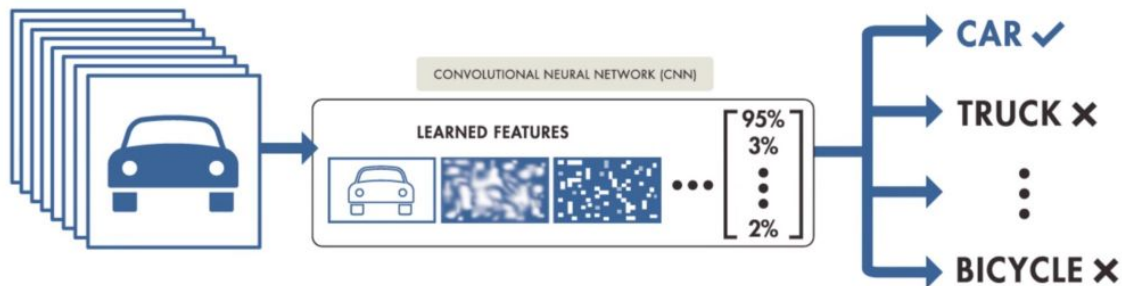
- Training with more data -> Data augmentation
- Ensembling
- Simplify the architecture: Limit the number of hidden layers and the number of units per layer.
- Early stopping
- Weight-decay: Penalize large weights using penalties or constraints on their squared values (L2 penalty) or absolute values (L1 penalty).
- Add noise to the weights or the activities.
- Dropout



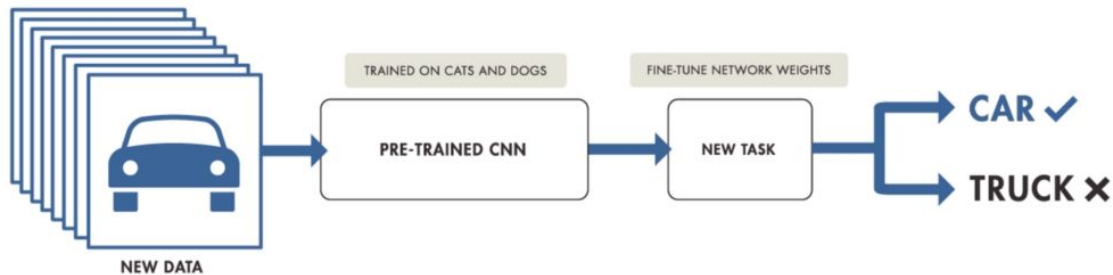
Typically, a combination of several of these methods is used.

Transfer learning

TRAINING FROM SCRATCH



TRANSFER LEARNING



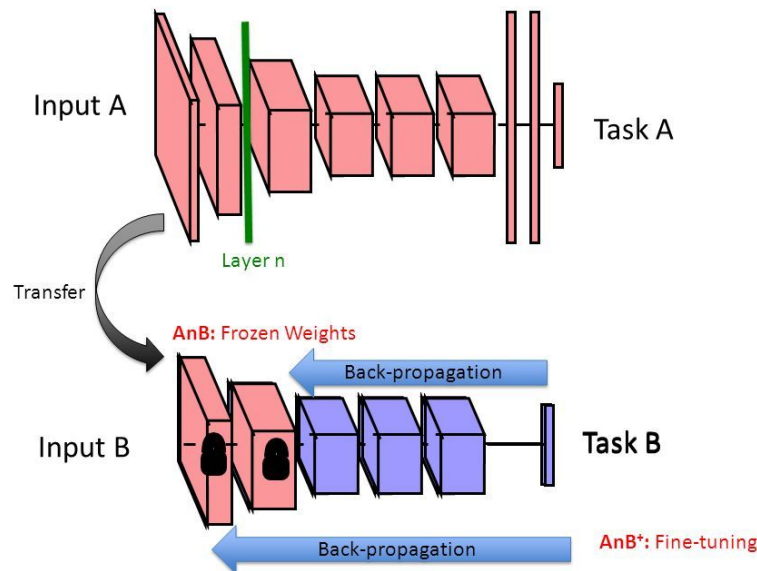
Freezing layers by transfer learning

Freezing layers in a CNN is a technique where some layers are kept fixed during training, meaning their weights are not updated.

This is a common practice in transfer learning to preserve features learned by a pre-trained model, reduce computational cost, and prevent overfitting.

Layers that capture general features are often frozen, while later layers that need to adapt to a specific new task are left unfrozen.

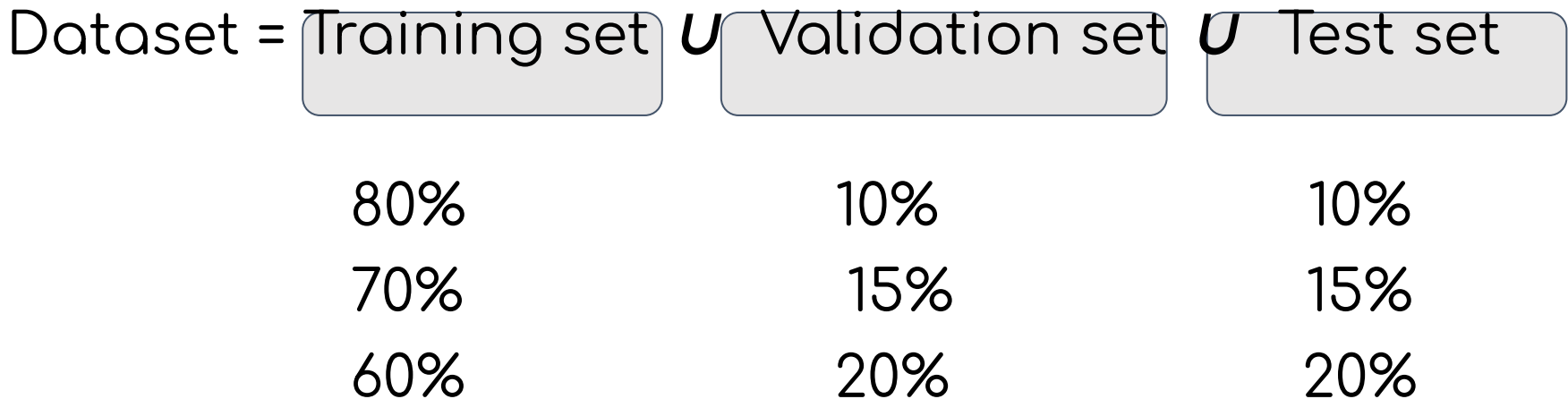
Transfer Learning Overview



Generic ML principles related to training data

Splitting data

A dataset is typically divided into three main subsets:



Splitting data

1. Training Set

used to set the model's parameters (weights and biases).

2. Validation Set

evaluation during the training

helps detect overfitting — when the model performs well on training data but poorly on unseen data.

3. Test Set

used after training is complete to provide an unbiased evaluation of the model's generalization performance.

The test set is never used for training or hyperparameter tuning.

Balancing training data

Balancing training data involves **adjusting the distribution of classes** in a dataset so that each class has a similar number of examples, preventing machine learning models from being biased toward the majority class.

Key techniques include:

- oversampling (duplicating or creating synthetic samples for the minority class),
 - undersampling (removing samples from the majority class),
 - using algorithmic approaches like cost-sensitive learning which assigns a higher weight to minority classes during training.
- using weighted loss function

Data augmentation

Data augmentation is a technique used to increase the number of data in the dataset without actually collecting new data.

Data Augmentation Basic transformations

- Geometric Transformations
 - Rotation, Flipping, Scaling, Translation, Shearing
- Color Space Augmentations
 - Color space augmentations modify the color properties of an image.
- Filtration, adding Noise
 - Kernel filters apply convolutional operations to enhance or suppress specific features in the image. It includes: Blurring, Sharpening...

Data Augmentation by Fully Synthetic Image Data

- AI-based image synthesis to generate entirely new samples- Generative NN
 - GAN-based Synthesis (Generative Adversarial Networks)
 - Diffusion Generative Models
- 3D Simulation-Based Synthesis
 - Uses rendering engines (e.g., Unity, Blender, CARLA) to generate labeled images under controlled environments.
 - Allows systematic variation of lighting, viewpoint, weather, and object poses.
 - Common in autonomous driving, robotics, and industrial inspection.

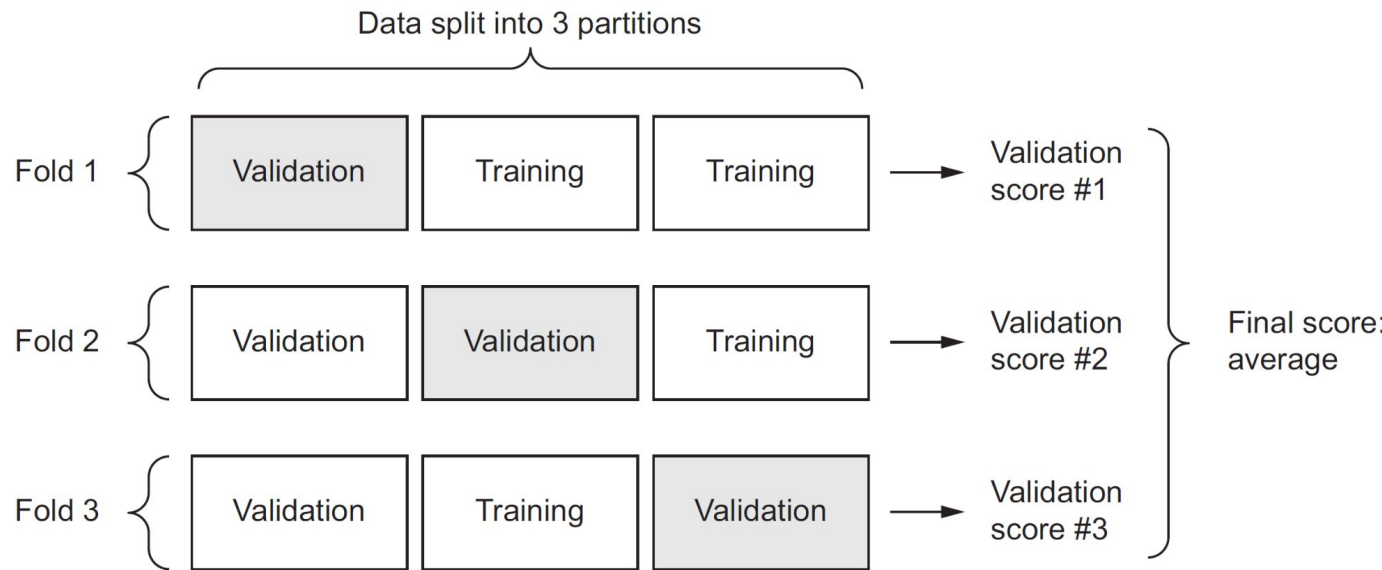
k-fold validation

If you have few data, the validation set would end up being very small. As a consequence, the validation scores might change a lot depending on which data you chose to use for validation and which you chose for training.

This would prevent you from reliably evaluating your model.

The best practice in such situations is to use K-fold cross-validation

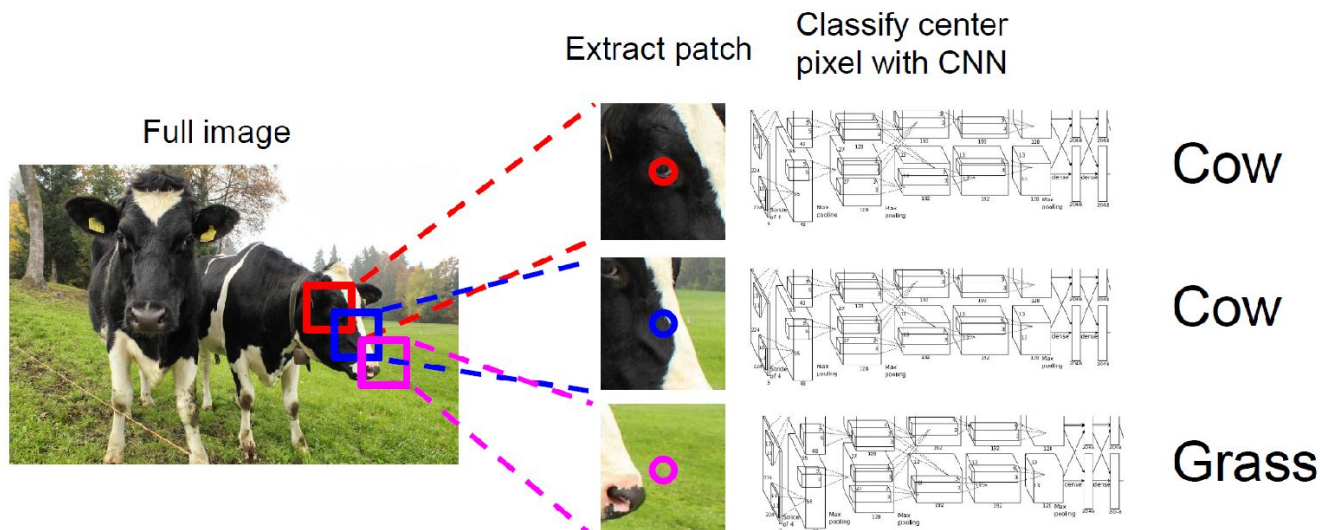
3-fold validation



Object segmentation using CNN

Semantic Segmentation Idea: Sliding Window

Problem: Very inefficient! Not reusing shared features between overlapping patches



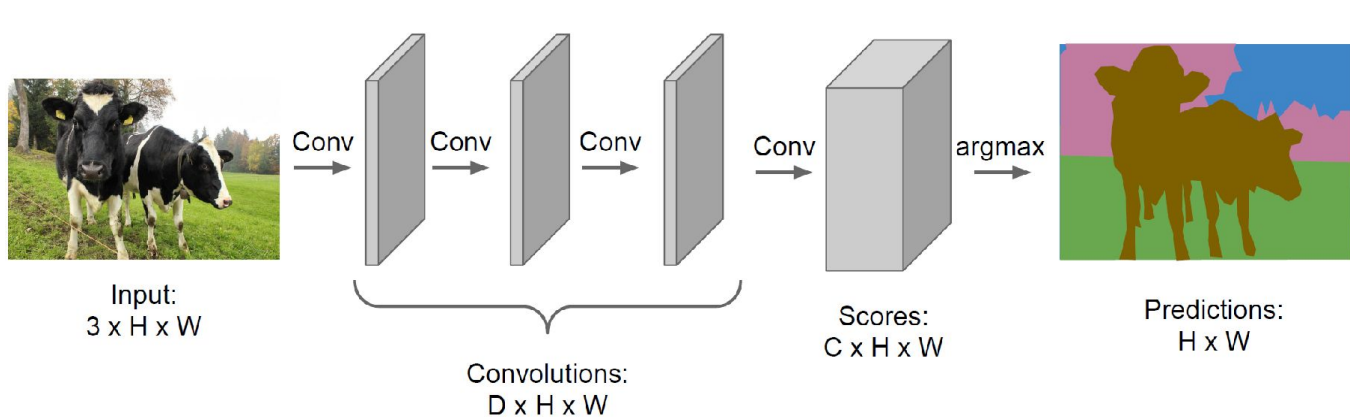
Farabet et al, "Learning Hierarchical Features for Scene Labeling," TPAMI 2013

Pinheiro and Collobert, "Recurrent Convolutional Neural Networks for Scene Labeling", ICML 2014

<http://cs231n.stanford.edu/slides/2019/>

Semantic Segmentation Idea: Fully Convolutional

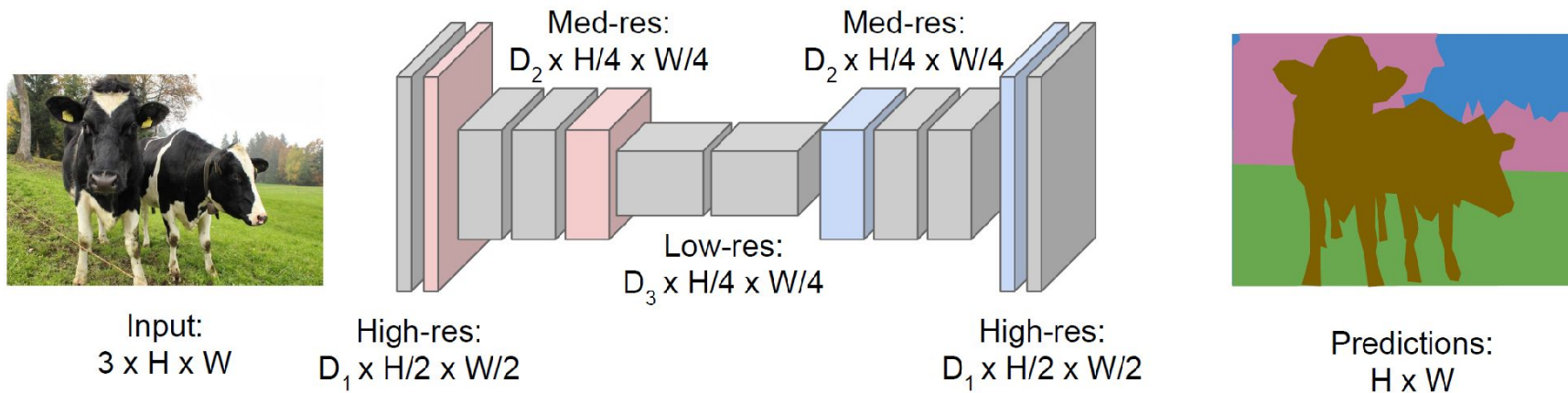
Design a network as a bunch of convolutional layers to make predictions for pixels all at once!



Problem: convolutions at original image resolution will be very expensive ...

Semantic Segmentation Idea: Fully Convolutional

Design network as a bunch of convolutional layers, with downsampling and upsampling inside the network!



Long, Shelhamer, and Darrell, "Fully Convolutional Networks for Semantic Segmentation", CVPR 2015

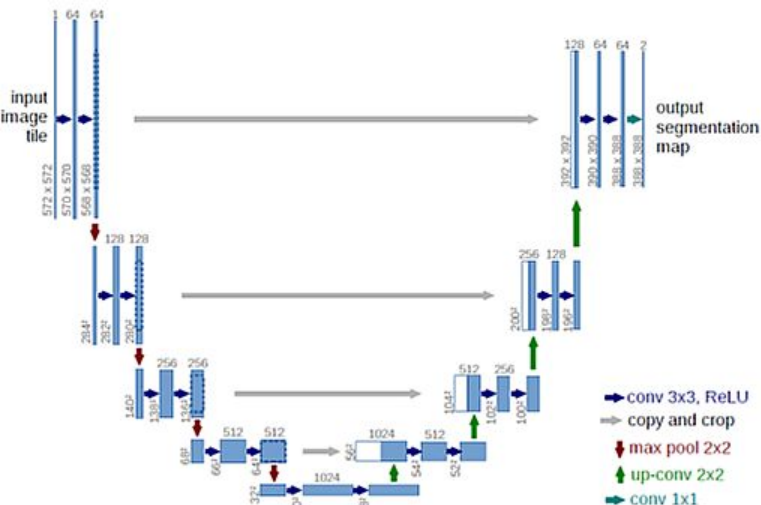
Noh et al, "Learning Deconvolution Network for Semantic Segmentation", ICCV 2015

<http://cs231n.stanford.edu/slides/2019>

U-net

U-Net architecture

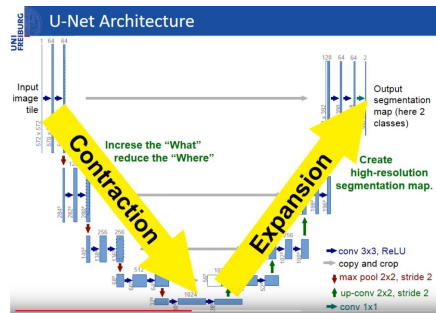
example for 32x32 pixels in the lowest resolution:



- Each blue box corresponds to a multi-channel feature map.
- The number of channels is denoted on top of the box.
- The x-y-size is provided at the lower left edge of the box.
- White boxes represent copied feature maps.
- The arrows denote the different operations.

U-Net architecture

It consists of
a contracting path (left side)
and
an expansive path (right side).



contracting path (left side)

Follows the typical architecture of a convolutional network:

- It consists of the repeated application of two 3x3 convolutions (unpadded convolutions), each followed by a rectified linear unit (ReLU) and a 2x2 max pooling operation with stride 2 for downsampling.
- At each downsampling step we double the number of feature channels.

expansive path (right side)

Every step in the expansive path consists of

- upsampling of the feature map followed by a 2x2 convolution ("up-convolution") that halves the number of feature channels,
- a concatenation with the correspondingly cropped feature map from the contracting path,
- and two 3x3 convolutions, each followed by a ReLU. The cropping is necessary due to the loss of border pixels in every convolution.
- At the final layer a 1x1 convolution is used to map each 64- component feature vector to the desired number of classes.

2D Transposed Convolution

„Deconvolution block“ used in Expansion

The 2D “Deconvolution block” represents a layer that performs an **opposite operation** to 2D Convolution.

The term „Deconvolution“ is not exact – NOT a Deconvolution in terms of signal theory

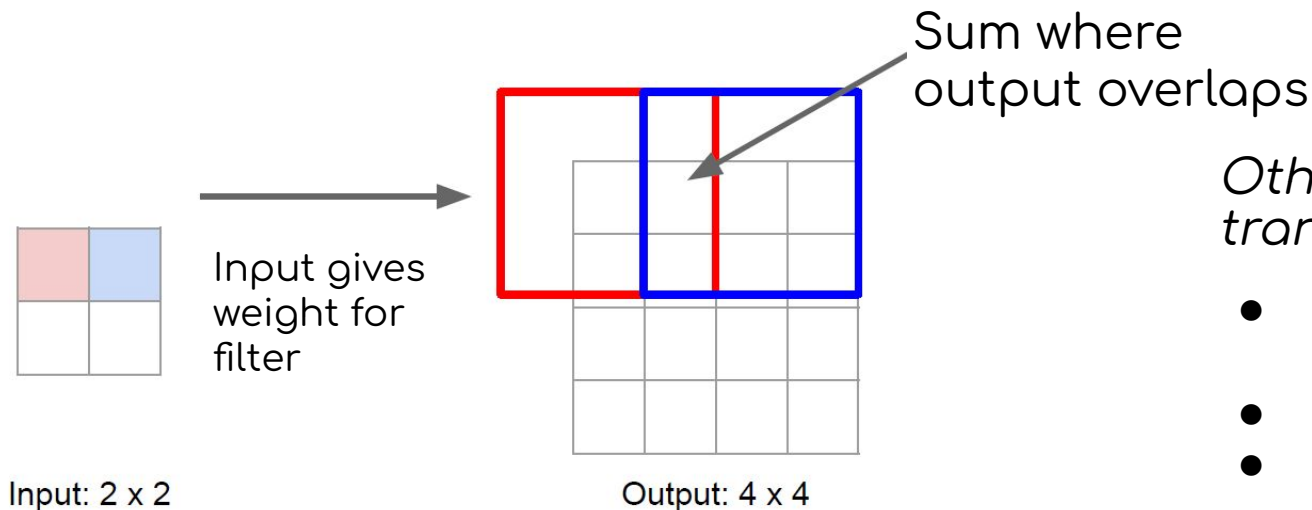
- Deconvolution blocks **have to learn weights** in the same way as Convolution layers
- operate similarly to convolution blocks, **but stride has the opposite effect** (making the output bigger, not smaller).

Usage: The deconvolution operation is used when we transform the output of a convolution back into a tensor that has the same shape as the input of that convolution.

Autoencoders and image segmentation models (U net) are examples of such models.

Upsampling: Transpose Convolution

3 x 3 transpose convolution, stride 2 pad 1

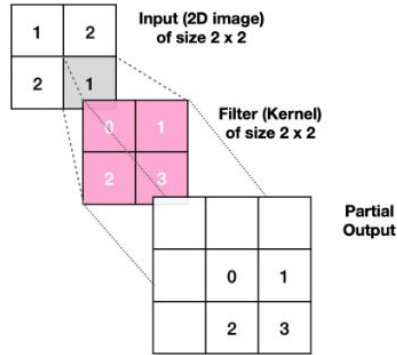


Other names of transpose convolution:

- Deconvolution (not exact)
- Upconvolution
- Fractionally strided convolution
- Backward strided convolution

2D Transposed Convolution - example

Transposed
convolution layer
with a 2x2 kernel.



0	1	
2	3	

 +

	0	2
	4	6

 +

0	2	
4	6	

 +

	0	1
	2	3

 =

=

0	1	2
2	9	7
4	8	3

Final
Output

U-Net Training

The **input images and their corresponding segmentation maps** are used to train the network with the stochastic gradient descent (SGD).

Loss function:

Binary cross-entropy loss — In the context of binary classification in neural networks, the cross-entropy loss $L(z,y)$ is commonly used and is defined as follows:

$$L(z,y) = -[y \cdot \log(z) + (1-y) \cdot \log(1-z)]$$

z : outputs predicted by the model, y : actual outputs

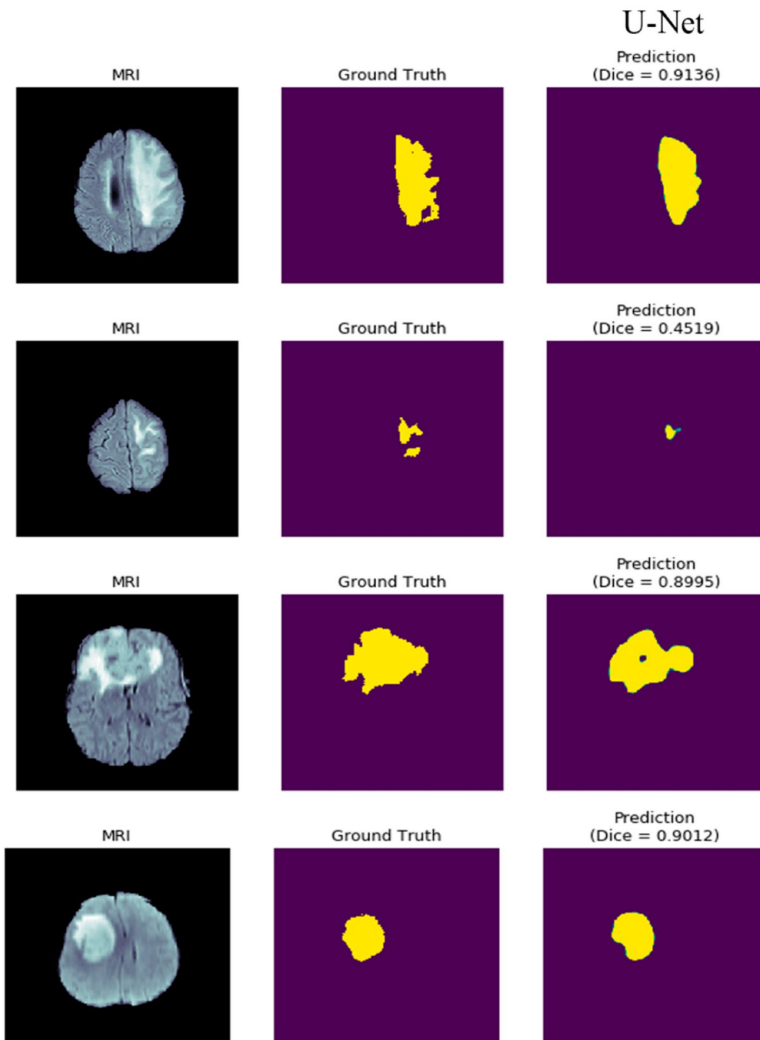
Dice loss:

One minus the Dice index

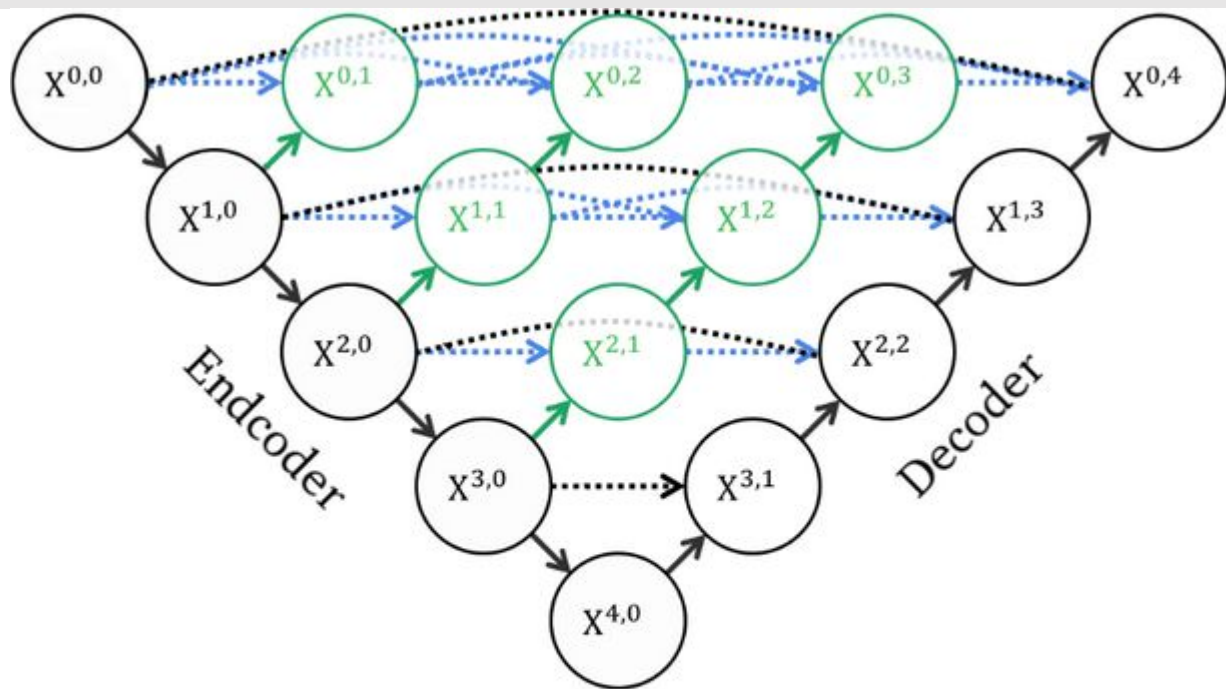
$$DICE = \frac{2 |S_g^1 \cap S_t^1|}{|S_g^1| + |S_t^1|} = \frac{2TP}{2TP + FP + FN}$$

Example:

Brain tumor segmentation using U-net



U-Net ++



$$x^{0,1} = H[x^{0,0}, U(x^{1,0})], \quad x^{0,2} = H[x^{0,0}, x^{0,1}, U(x^{1,1})], \quad \dots$$

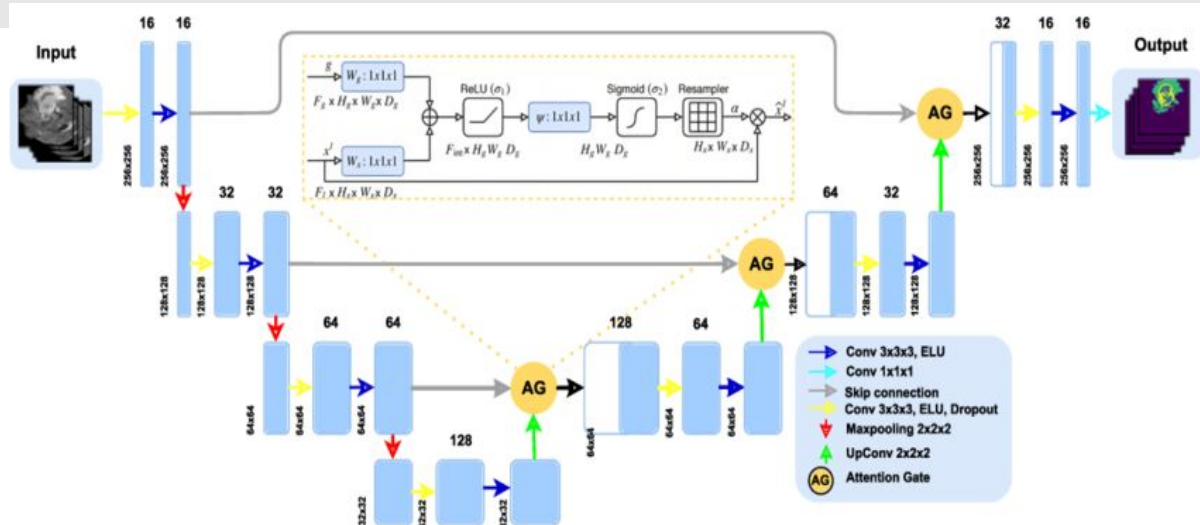


Down-sampling (2*2)



Skip connection

Attention U-net



The attention gate AG involves two input vectors: X and G .

Vector G is derived from the lower layer of the network and has smaller dimensions but better feature representation.

Vector X undergoes a strided convolution, while vector G undergoes a 1×1 convolution.

These vectors are then combined through element-wise summation, resulting in a new vector. This new vector is passed through activation and convolutional layers to reduce its dimensions. A sigmoid layer is then applied to produce attention coefficients, which represent the importance of each element in vector X .

To restore the attention coefficients to the original dimensions, trilinear interpolation is used. The attention coefficients are multiplied with the original vector X , scaling it based on relevance. Finally, the modified vector is passed through the skip connection for further processing.